539.11
Рейтинг
Подписаться

Яндекс
Как мы делаем Яндекс

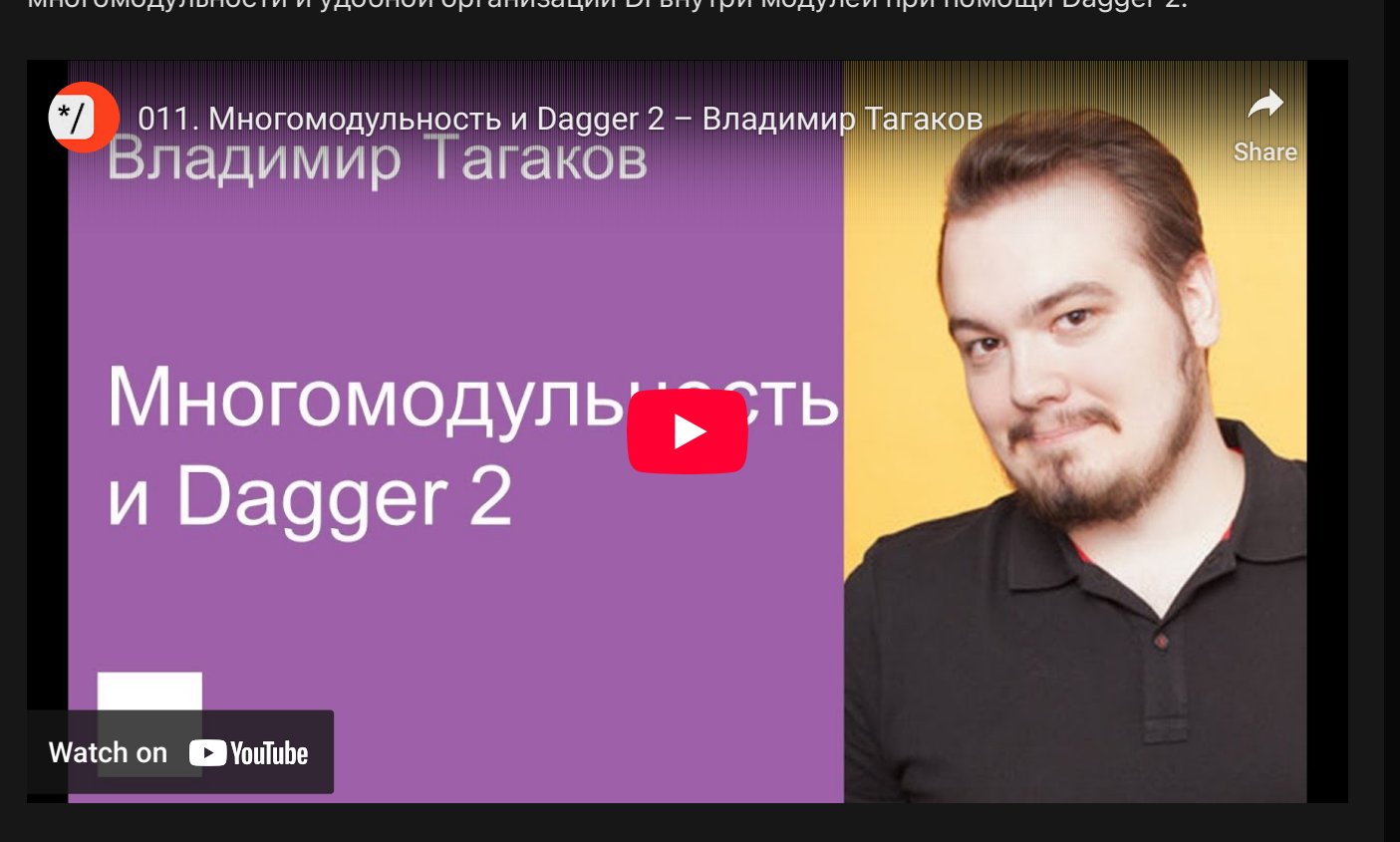
Leono 5 авг 2018 в 17:32

Многомодульность и Dagger 2. Лекция Яндекса

9 мин 21K

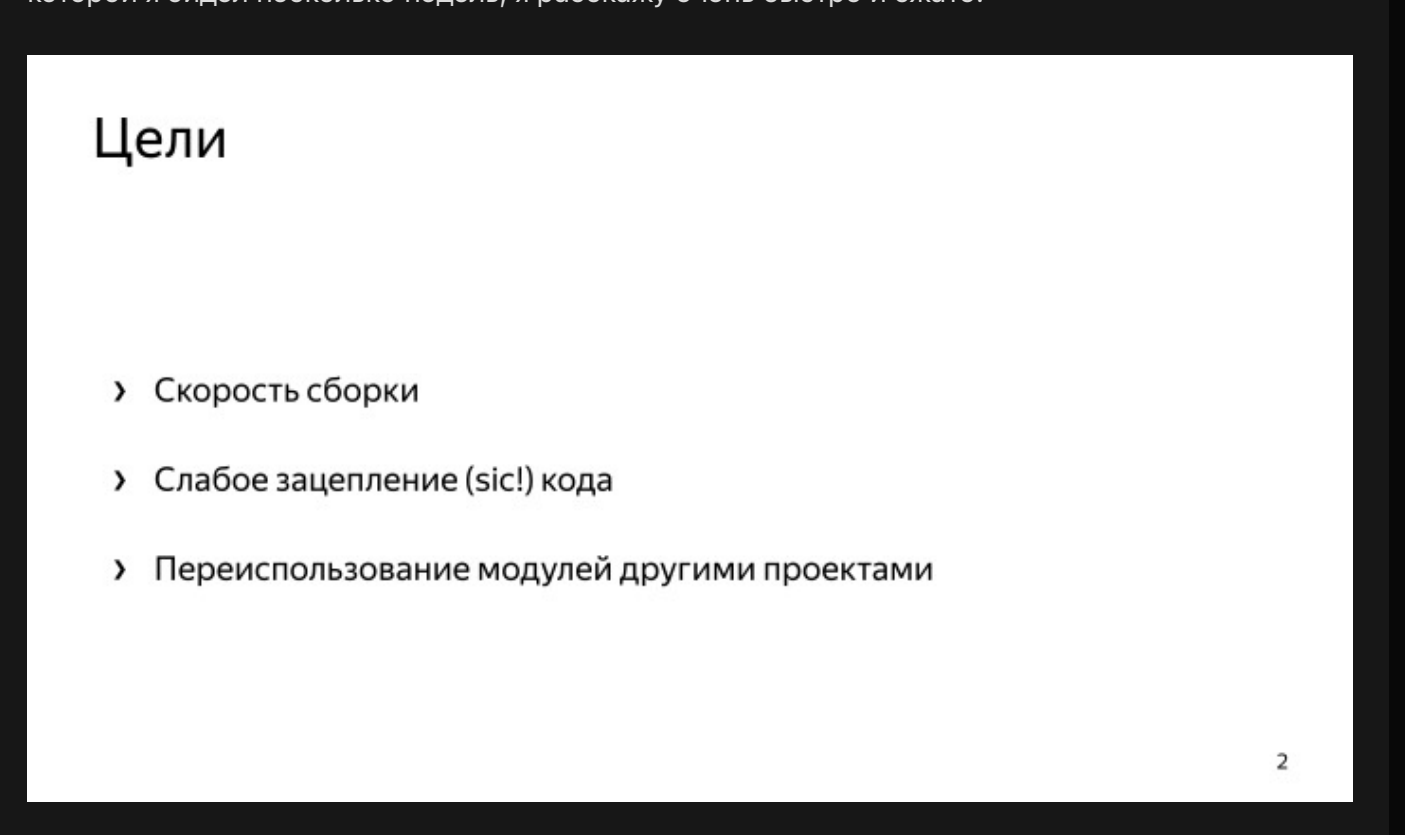
Блог компании Яндекс, Разработка мобильных приложений*, Разработка под Android*

Когда ваше приложение построено на многомодульной архитектуре, приходится посвящать много времени тому, чтобы все связи между модулями были корректно прописаны в коде. Половину этой работы можно поручить фреймворку Dagger 2. Руководитель группы Яндекс.Карт для Android Владимир Тагаков @Noxa рассказал о плюсах и минусах многомодульности и удобной организации DI внутри модулей при помощи Dagger 2.


011. Многомодульность и Dagger 2 – Владимир Тагаков
Владимир Тагаков
Многомодульность и Dagger 2
Share
Watch on YouTube

— Меня зовут Владимир, я разрабатываю Яндекс.Карты и сегодня буду рассказывать вам про модульность и второй Dagger.

Самую длинную часть я понимал, когда сам ее изучал, быстрее всего. Вторую часть, над которой я сидел несколько недель, я расскажу очень быстро и сжато.

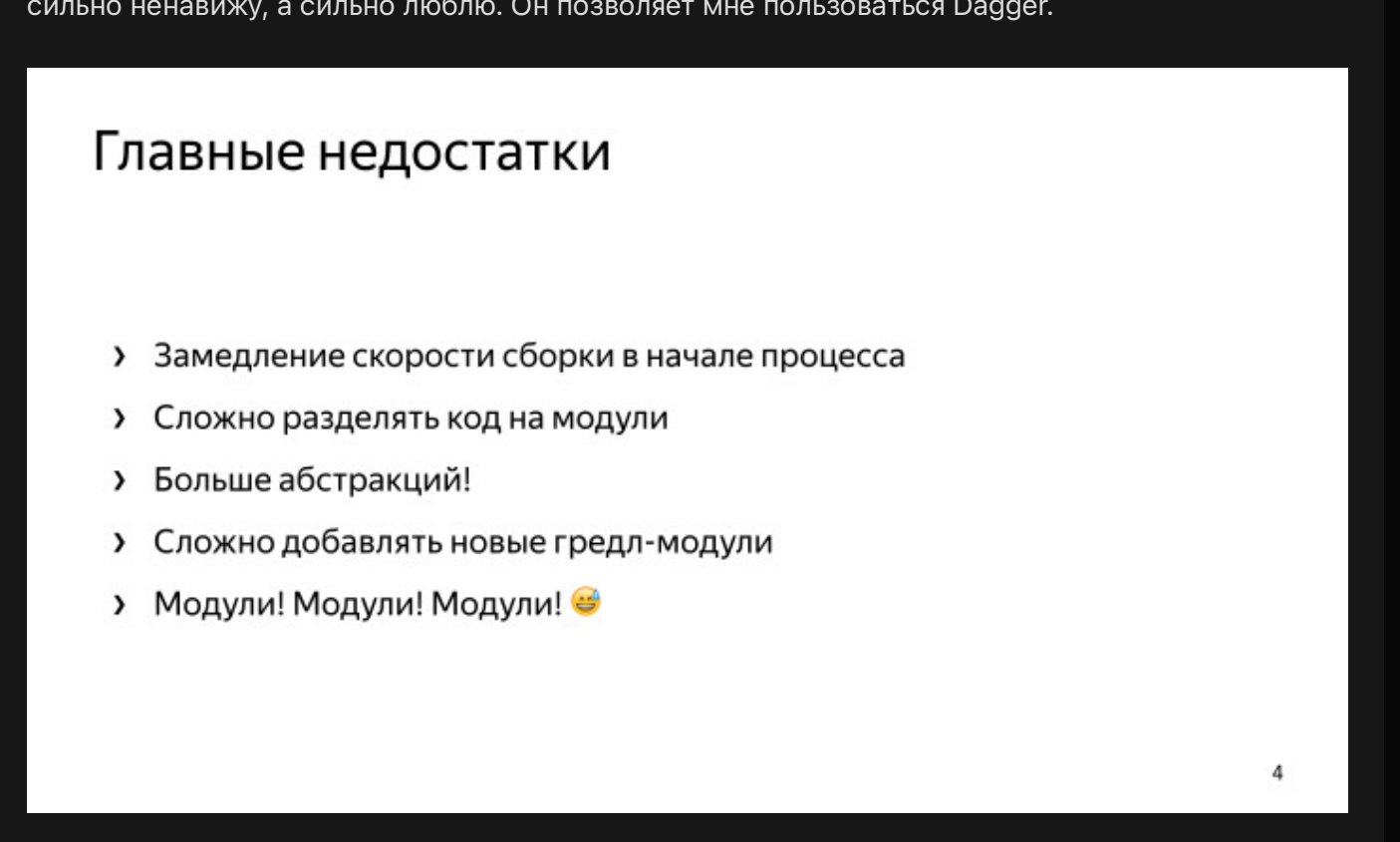

Цели

- Скорость сборки
- Слабое зацепление (sic!) кода
- Переиспользование модулей другими проектами

Зачем мы в Картах начали непростой процесс разделения на модули? Мы хотели просто повысить скорость сборки, все про это знают.

Второй пункт цели — уменьшить зацепление кода. Зацепление я взял из Википедии. Это значит, что мы хотели уменьшить взаимосвязи между модулями, чтобы модули были обособленными и могли использоваться вне приложения. Изначальная постановка задачи: другие проекты Яндекса должны иметь возможность использовать часть функциональности Карт ровно так, как у нас. И чтобы разработкой этой функциональности занимались мы в рамках развития проекта.

Хочу бросить горящий тапок в сторону [k]apt, который замедляет скорость сборки. Я его не сильно ненавижу, а сильно люблю. Он позволяет мне пользоваться Dagger.


Главные недостатки

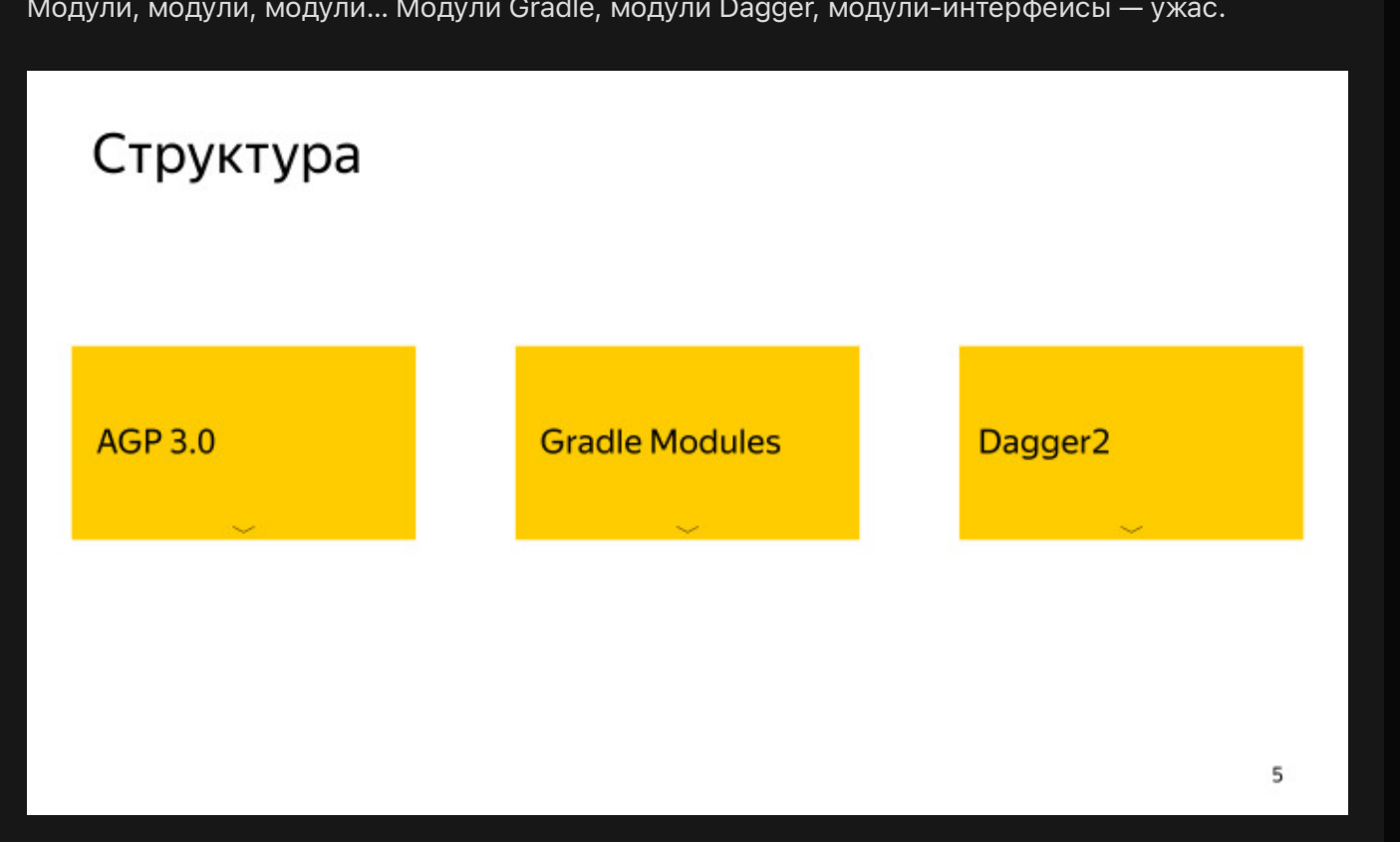
- Замедление скорости сборки в начале процесса
- Сложно разделять код на модули
- Больше абстракций!
- Сложно добавлять новые гредл-модули
- Модули! Модули! Модули! 😅

Главный недостаток процесса разделения на модули — это, как ни парадоксально, замедление скорости сборки. Особенно в самом начале, когда вы выносите первые два модуля, Common и какую-то вашу фичу, общая скорость сборки проекта падает, как бы вы ни старались. В конце, когда в вашем главном модуле будет оставаться все меньше кода, скорость сборки будет расти. И все равно это не значит, что все очень плохо, есть способы это обойти и даже из первого модуля поиметь профит.

Второй недостаток — сложно разделять код на модули. Кто пытался, знает, что ты начинаешь тянуть какие-то зависимости, какие-то классики, и все кончается тем, что ты весь свой главный модуль скопировал в другой модуль и начинаешь заново. Поэтому нужно четко понимать момент, когда нужно остановиться и разбить связь, используя какую-то абстракцию. Недостаток — больше абстракций. Больше абстракций — сложнее проект — больше абстракций.

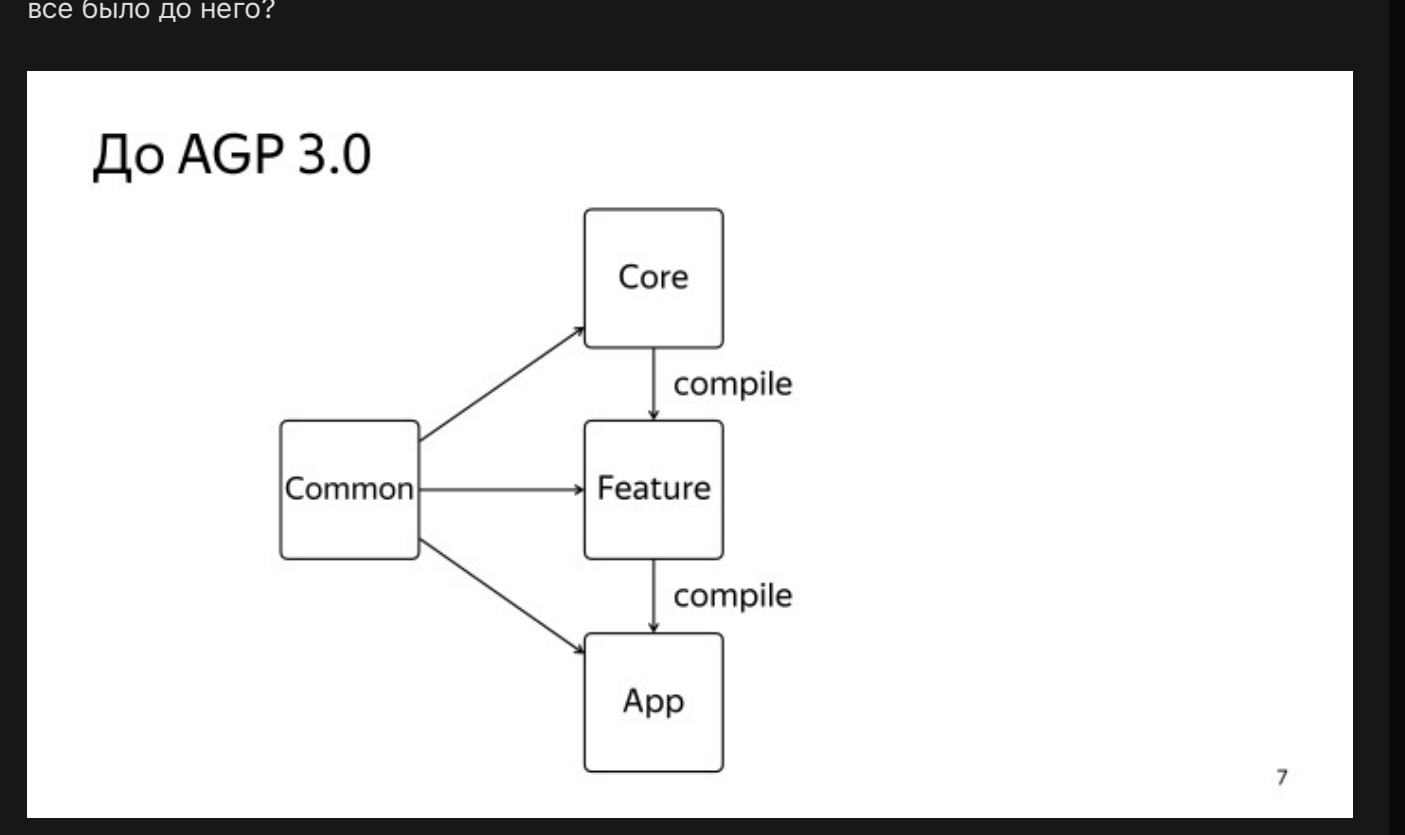
Сложно добавлять новые Gradle-модули. Почему? Например, приходит разработчик, берет в разработку новую фичу, сразу делает хорошо, делает отдельный модуль. В чем проблема? Он должен помнить обо всем доступном коде, который есть в главном модуле, чтобы, если что, его переиспользовать и вынести в Common. Потому что процесс выноса какого-то модуля в Common — постоянный, пока ваш главный модуль App не превратится в тонкую прослойку.

Модули, модули, модули… Модули Gradle, модули Dagger, модули-интерфейсы — ужас.


Структура

AGP 3.0 Gradle Modules Dagger2

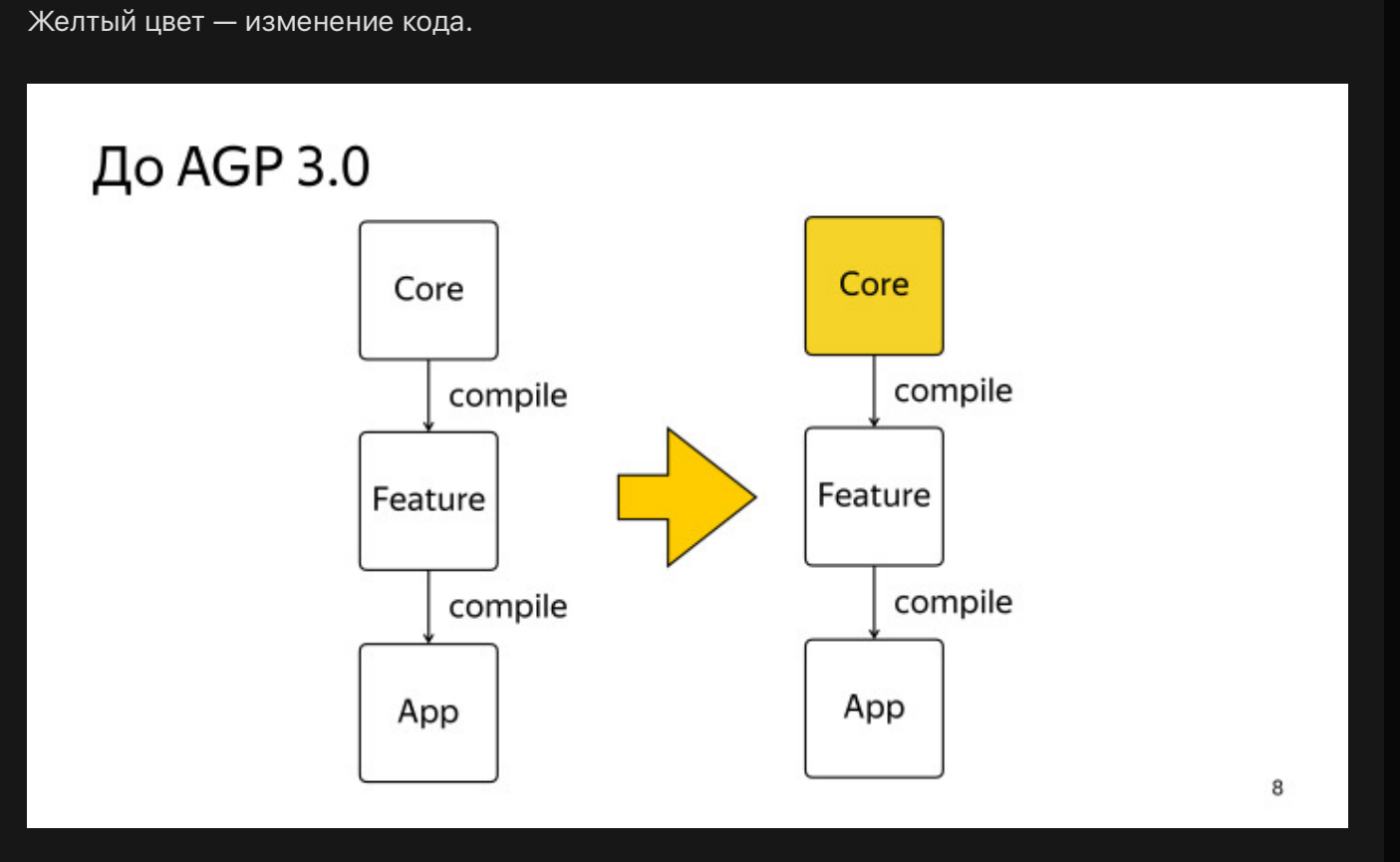
Доклад будет состоять из трех частей: маленькой, большой и сложной. Вначале про разницу между Implementation и API в AGP. Android Gradle Plugin 3.0 появился относительно недавно. Как все было до него?


До AGP 3.0
Core
compile
Common
Feature
compile
App
7

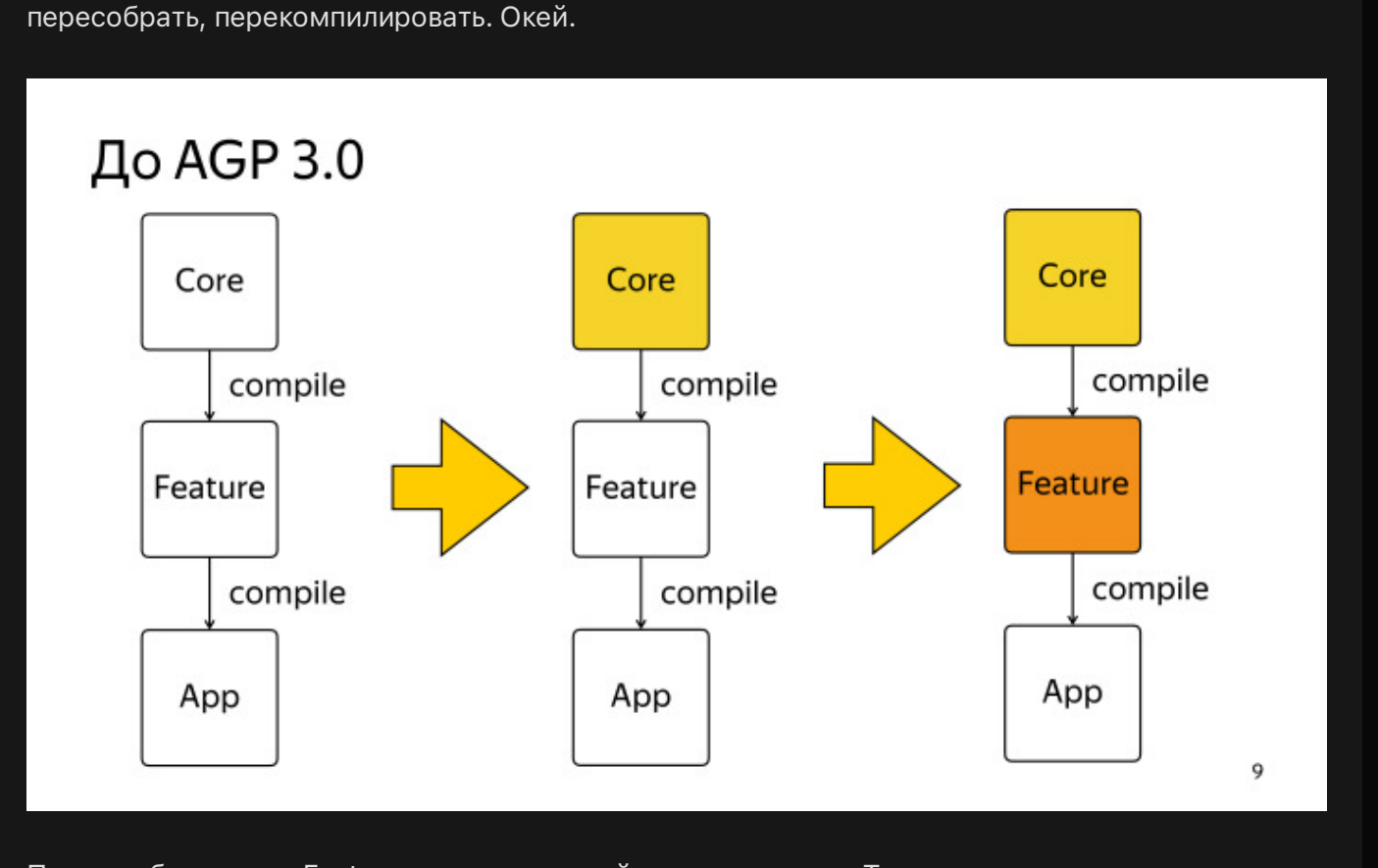
Вот типичный проект здорового разработчика, состоящий из трех модулей: модуль App, который является главным, собирается и устанавливается в приложение, и два Feature-модуля.

Сразу поговорим про стрелки. Это большая боль, каждый рисует в ту сторону, в которую ему удобно рисовать. У меня они значат, что от Core идет стрелка к Feature. Значит, Feature знает о Core, может пользоваться классами из Core. Как видите, между Core и App стрелки нет, значит App вроде как не должен использовать Core. Core — не Common-модуль, он есть, от него все зависят, он отдельный, в нем мало кода. Пока его не будем рассматривать.

Наш Core-модуль изменился, нам нужно как-то его переделать. Мы изменяем в нем код. Желтый цвет — изменение кода.

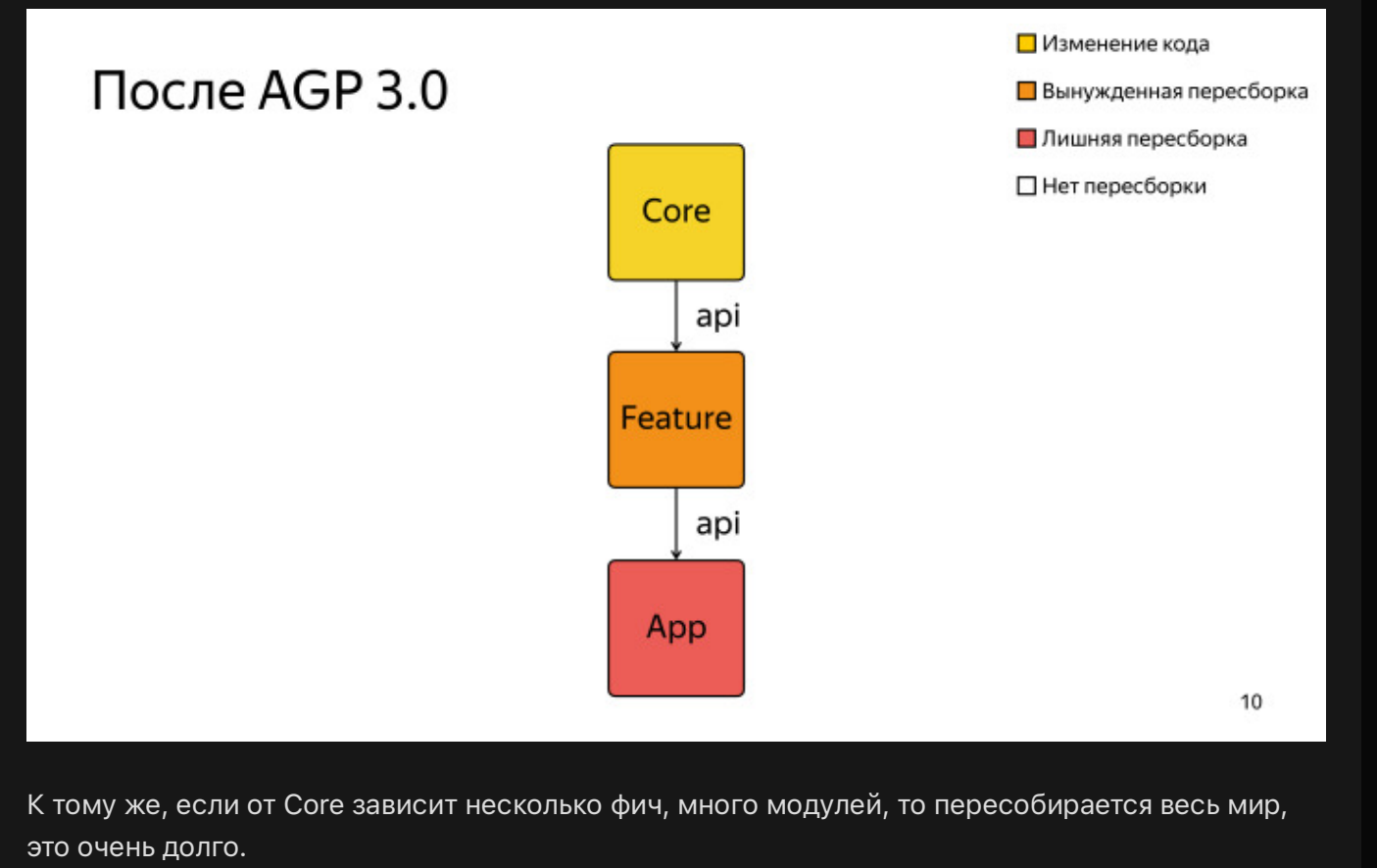

До AGP 3.0
Core
compile
Feature
compile
App
Core
compile
Feature
compile
App
8

После пересобираем проект. Понятно, что после изменения какого-то модуля его придется пересобрать, перекомпилировать. Окей.


До AGP 3.0
Core
compile
Feature
compile
App
Core
compile
Feature
compile
App
Core
compile
Feature
compile
App
9

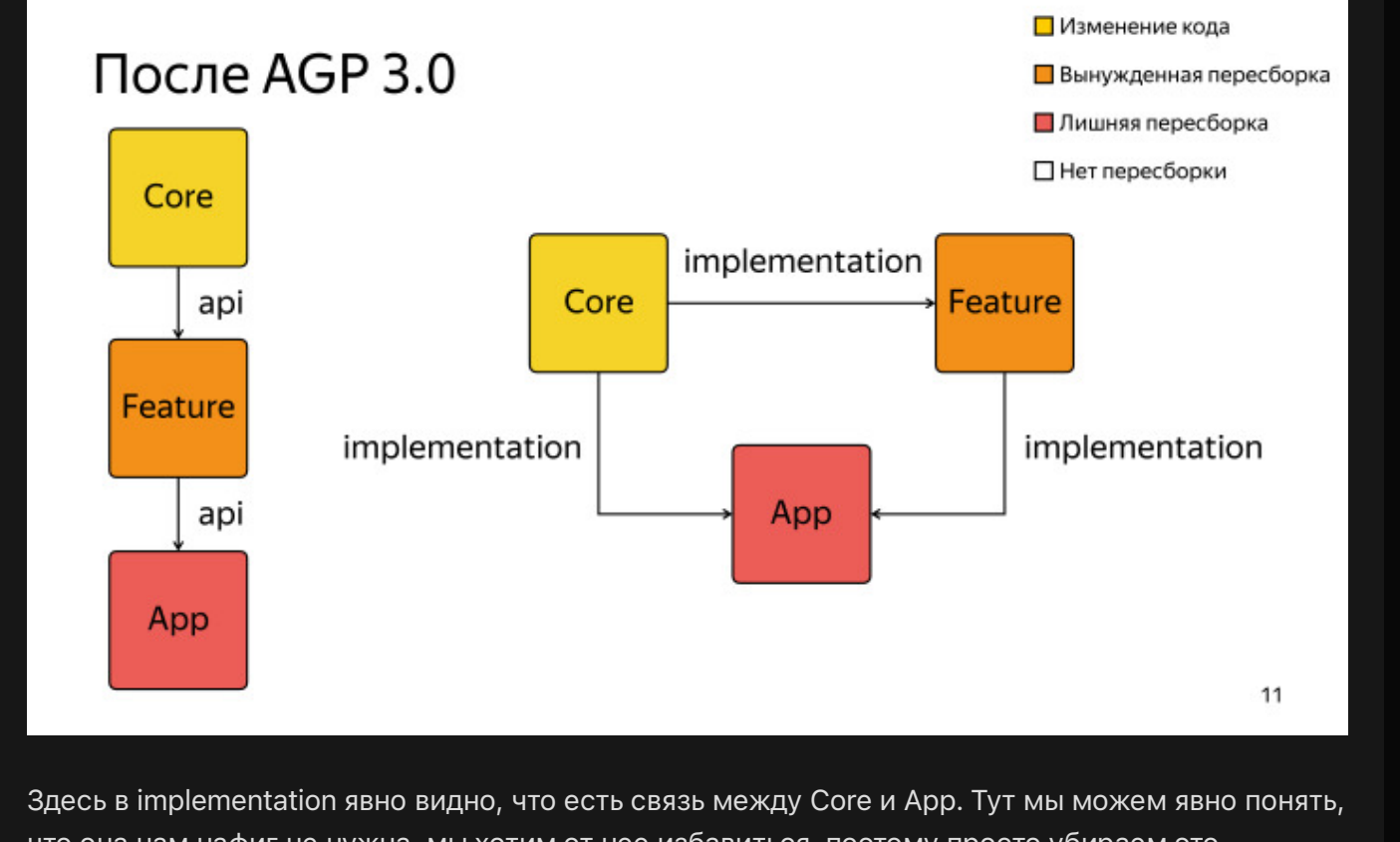
После собирается и Feature модуль, который от него зависит. Тоже понятно, его зависимость пересобралась, и нужно себя обновить. Кто знает, что там изменилось.

И тут происходит самое неприятное. Собирается модуль App, хотя непонятно почему. Я точно знаю, что Core я никак не использую, и почему App пересобирается — неясно. А он очень большой, потому что в самом начале пути, и это очень большая боль.

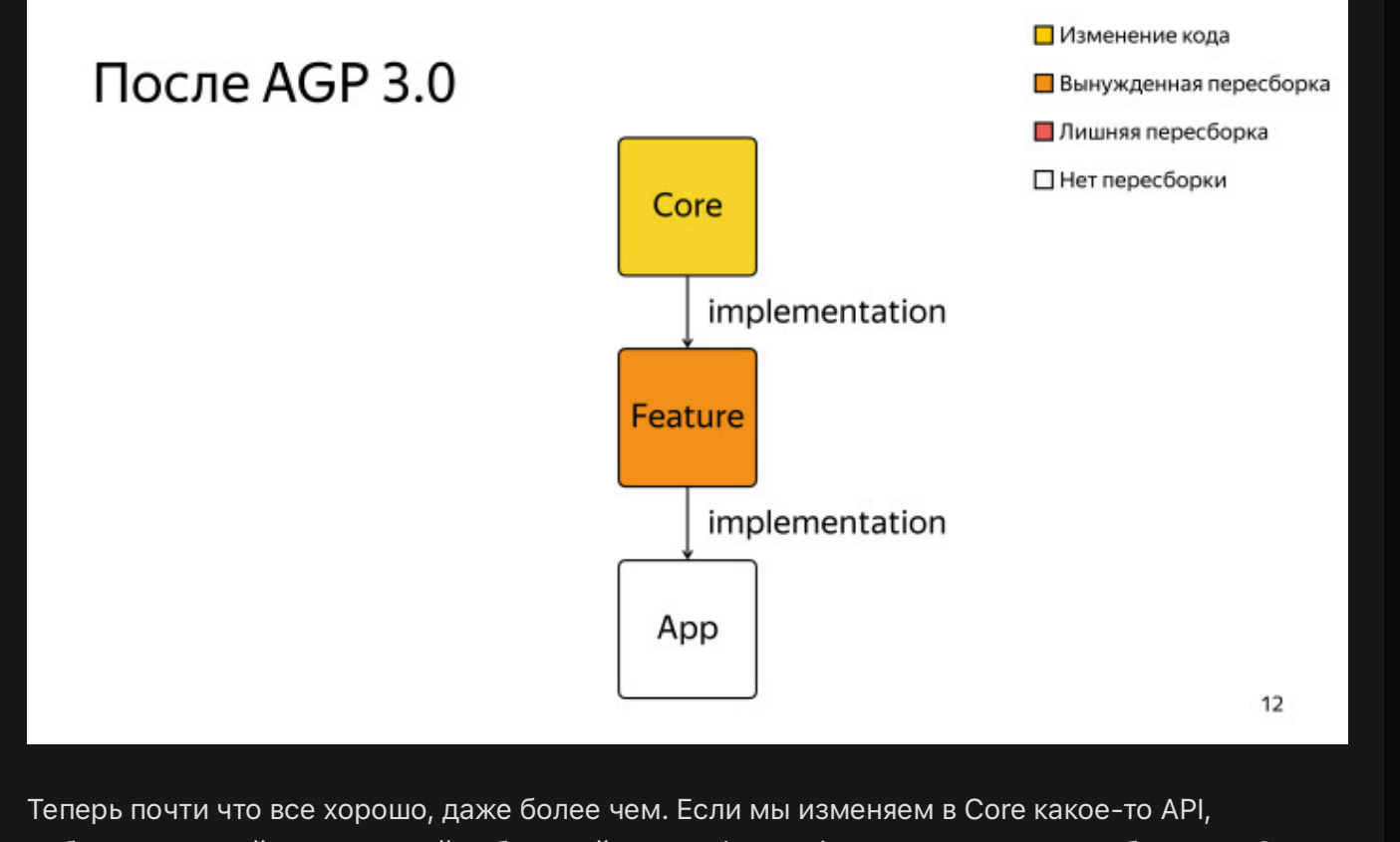

После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
api
Feature
api
App
10

К тому же, если от Core зависит несколько фич, много модулей, то пересобирается весь мир, это очень долго.

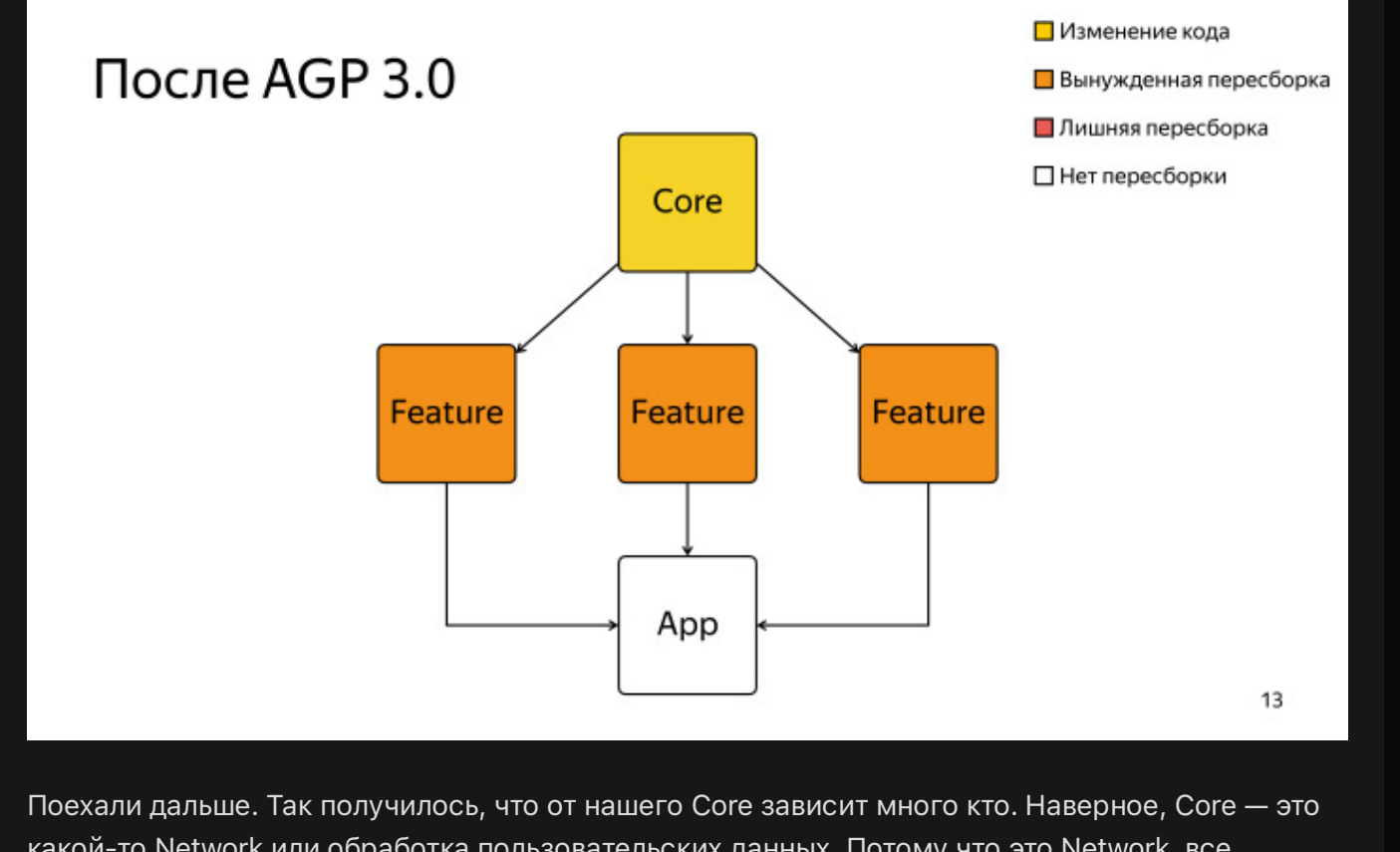
Давайте перейдем на новую версию AGP и заменим, как говорит инструкция, все compile на API, а не на Implementation, как вы подумали. Ничего не меняется. Схемы тождественные. Что за новый способ указывания зависимостей Implementation? Представим эту же схему, используя только это ключевое слово, без API? Оно будет выглядеть так.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
api
Feature
api
App
Core
implementation
Feature
implementation
App
implementation
11

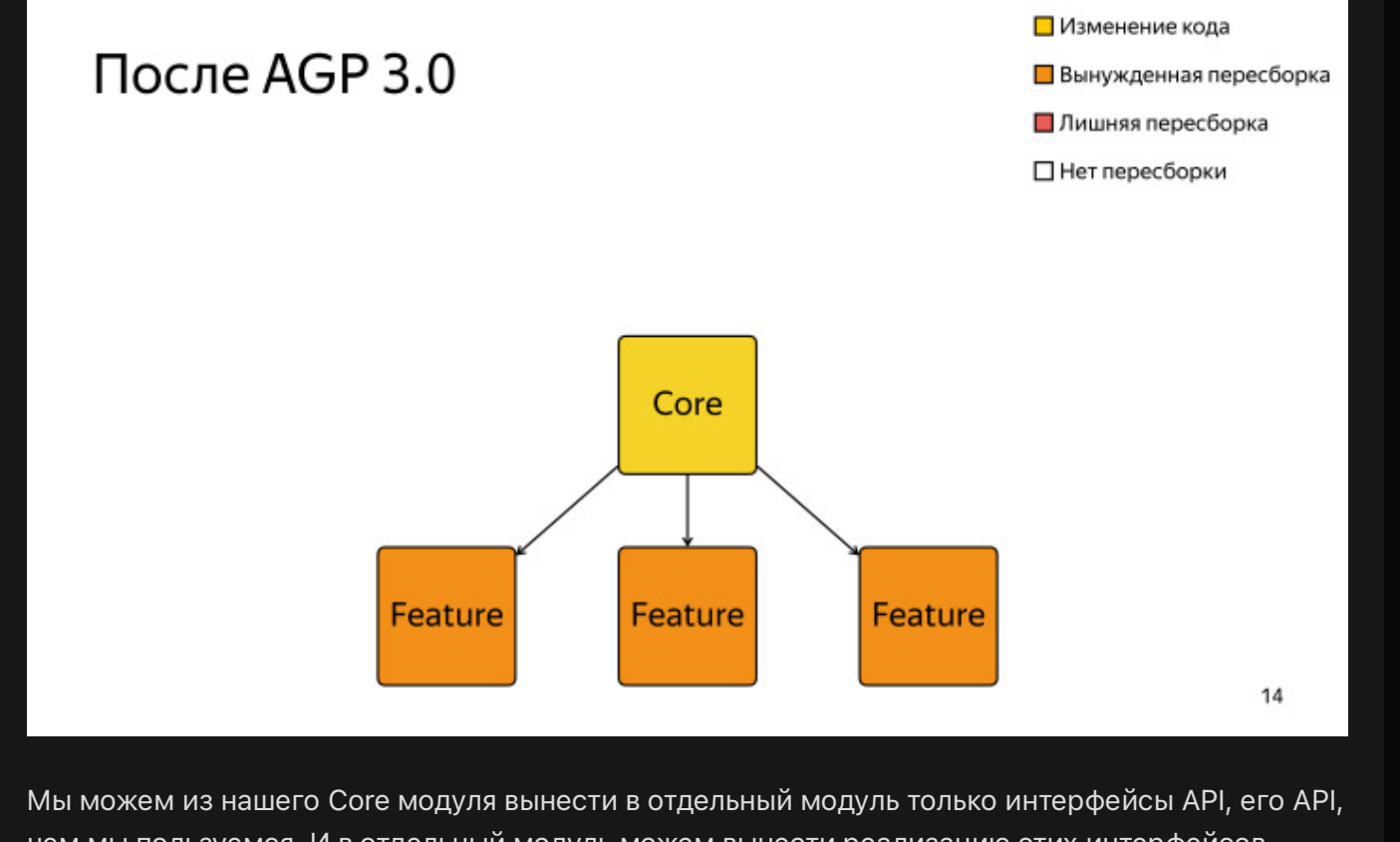
Здесь в implementation явно видно, что есть связь между Core и App. Тут мы можем явно понять, что она нам нафиг не нужна, мы хотим от нее избавиться, поэтому просто убираем это. Становится все вроде попроще.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
implementation
Feature
implementation
App
12

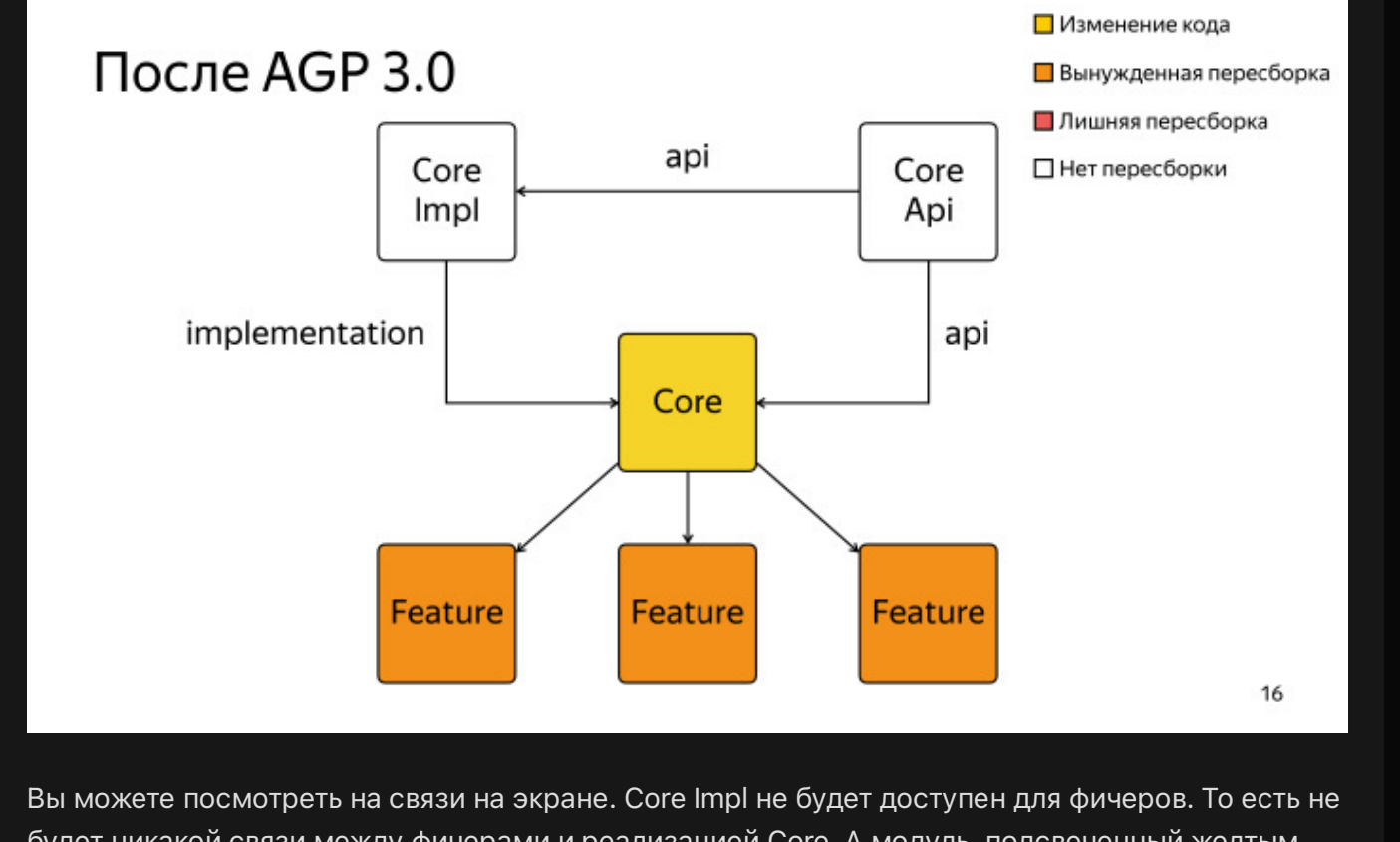
Теперь почти что все хорошо, даже более чем. Если мы изменяем в Core какое-то API, добавляем новый класс, новый публичный или package private метод, то пересобирается Core и Feature. Если меняете внутри метода реализацию или добавляете приватный метод, то теоретически пересборки Feature вообще ничего не должно происходить, потому что ничего же не изменилось.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
Feature
Feature
Feature
App
13

Поехали дальше. Так получилось, что от нашего Core зависит много кто. Наверное, Core — это какой-то Network или обработка пользовательских данных. Потому что это Network, все меняется довольно часто, все пересобирается, и мы получили ту же боль, от которой старательно убегали.
Давайте рассмотрим два способа, как можно с этим бороться.

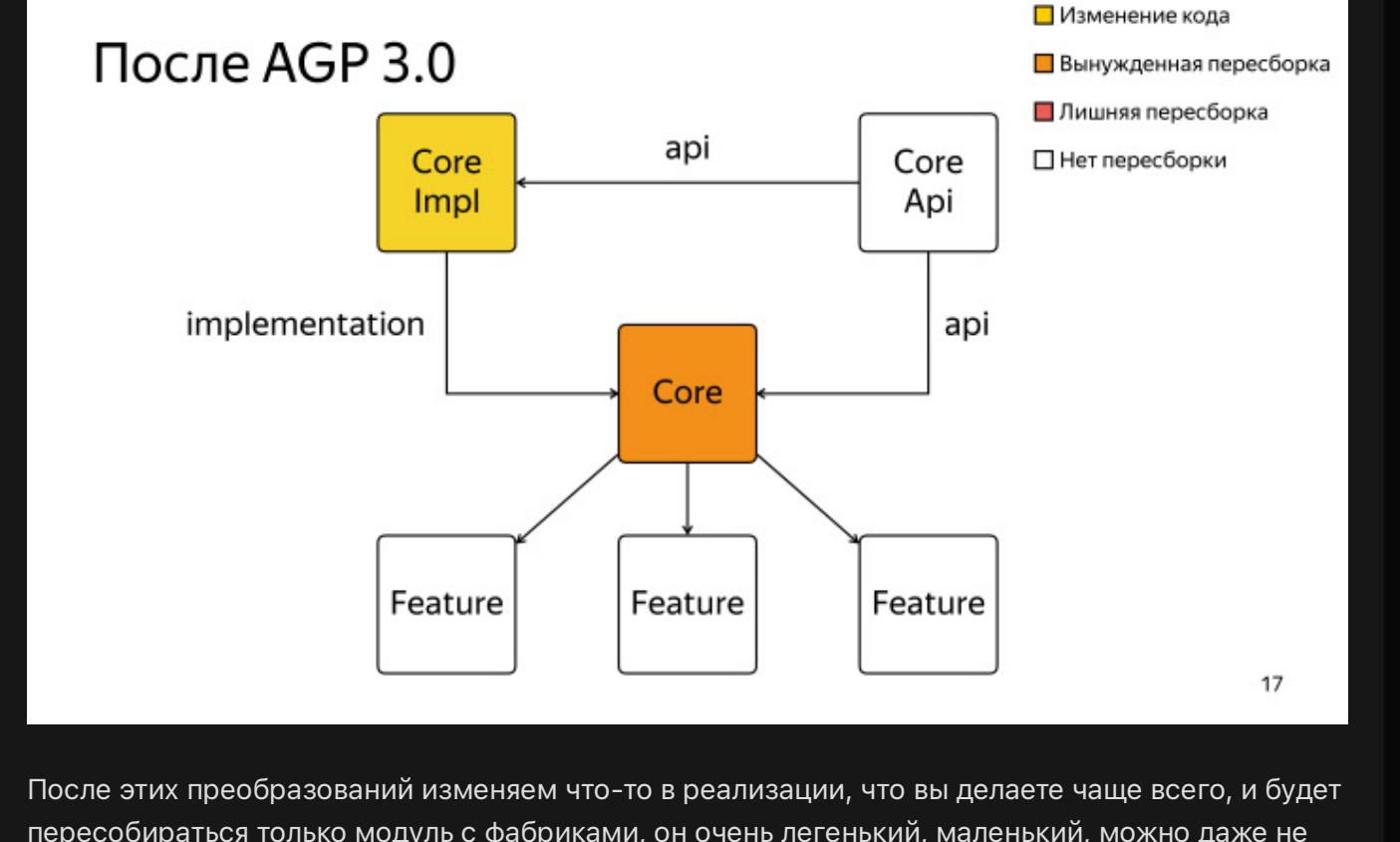

После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
Feature
Feature
Feature
14

Мы можем из нашего Core модуля вынести в отдельный модуль только интерфейсы API, его API, чем мы пользуемся. И в отдельный модуль можем вынести реализацию этих интерфейсов.

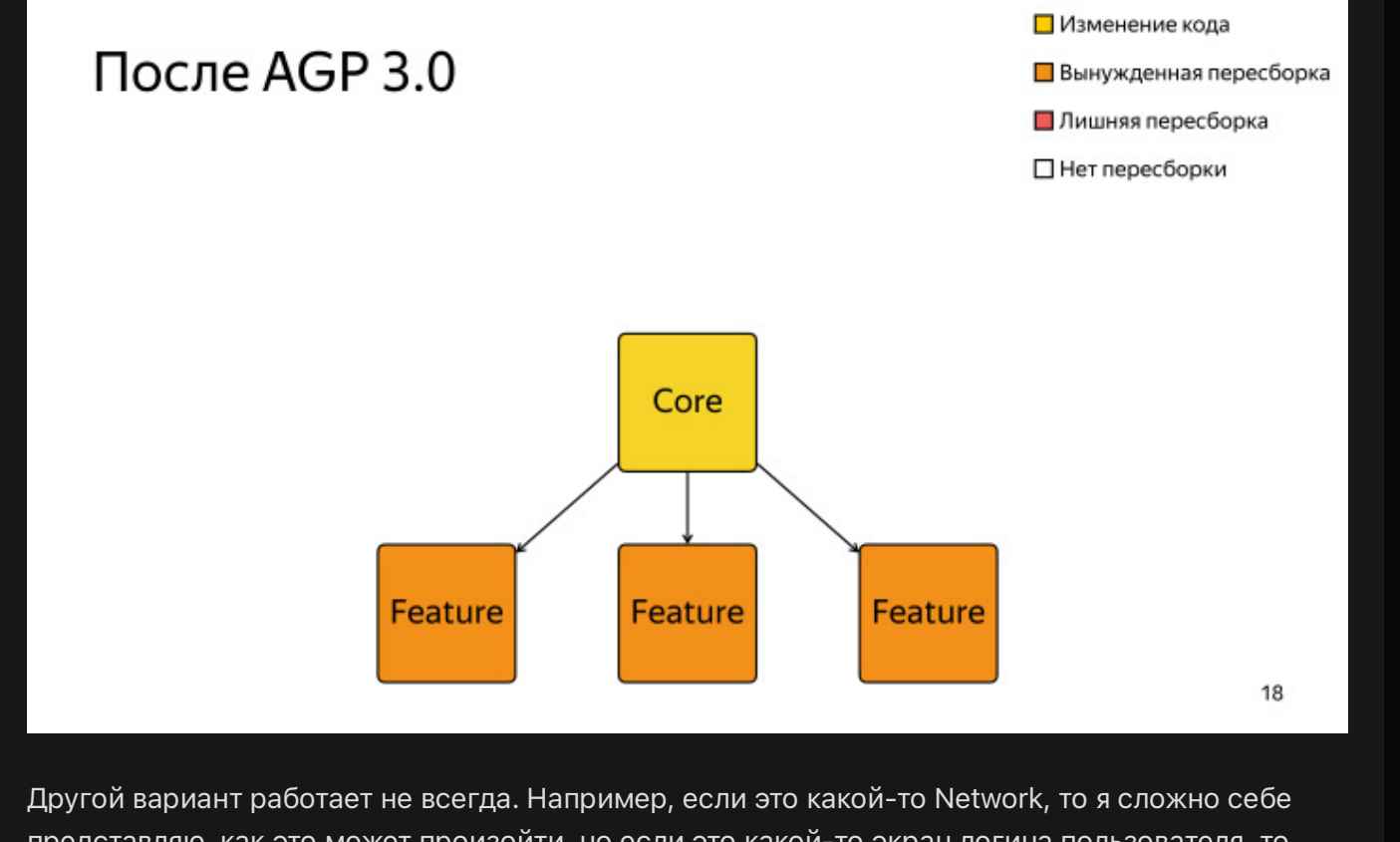

После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core Impl
api
Core Api
implementation
api
Core
Feature
Feature
Feature
16

Вы можете посмотреть на связи на экране. Core Impl не будет доступен для фичеров. То есть не будет никакой связи между фичерами и реализацией Core. А модуль, подсвеченный желтым, будет только предоставлять фабрики, которые будут предоставлять какие-то никому не известные реализации ваших интерфейсов.

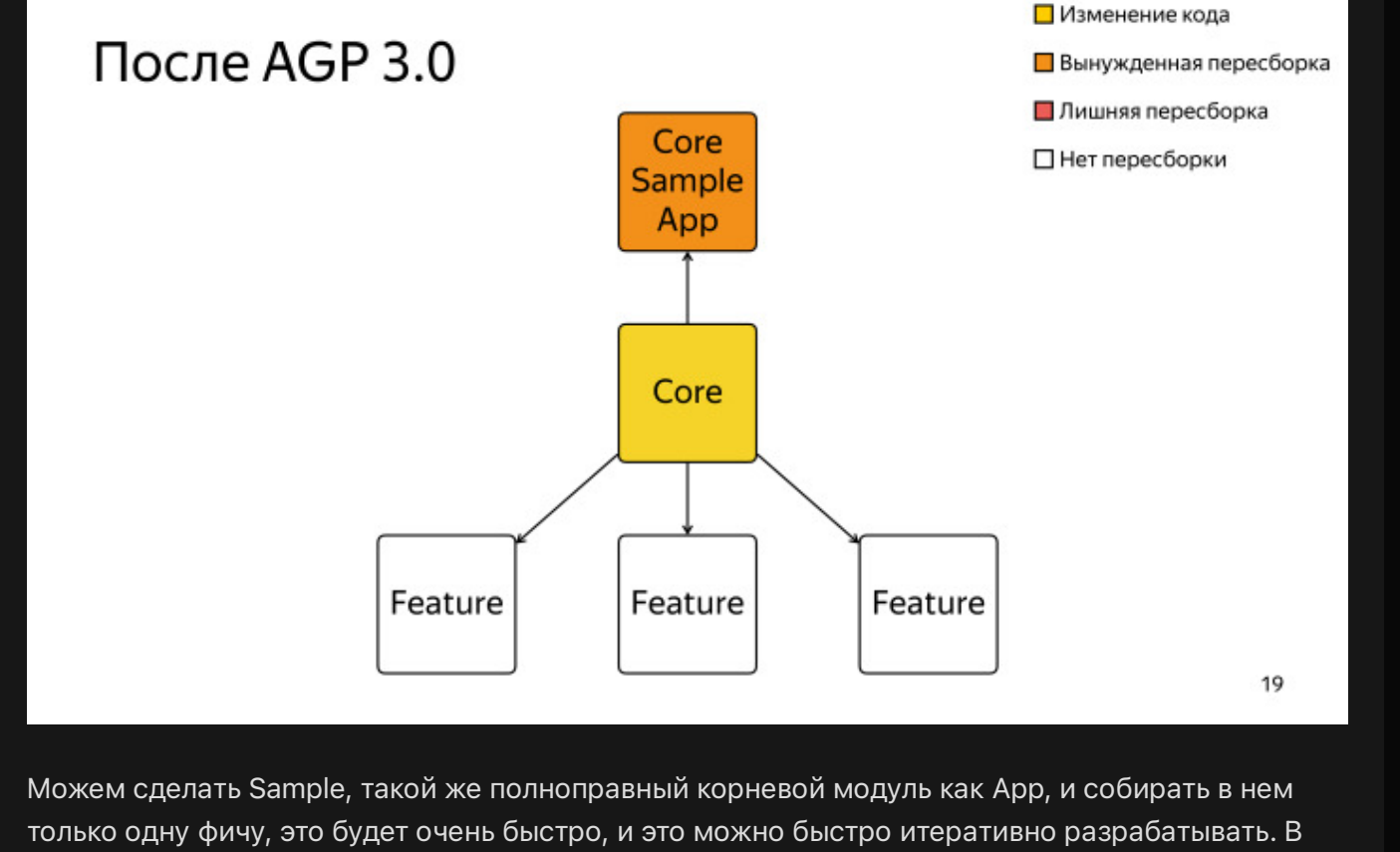
После такого преобразования, хочу обратить внимание, что Core API, из-за того что ключевое слово API стоит, будет доступен всем фичерам транзитивно.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core Impl
api
Core Api
implementation
api
Core
Feature
Feature
Feature
17

После этих преобразований изменяем что-то в реализации, что вы делаете чаще всего, и будет пересобираться только модуль с фабриками, он очень легенький, маленький, можно даже не рассматривать, сколько времени это занимает.

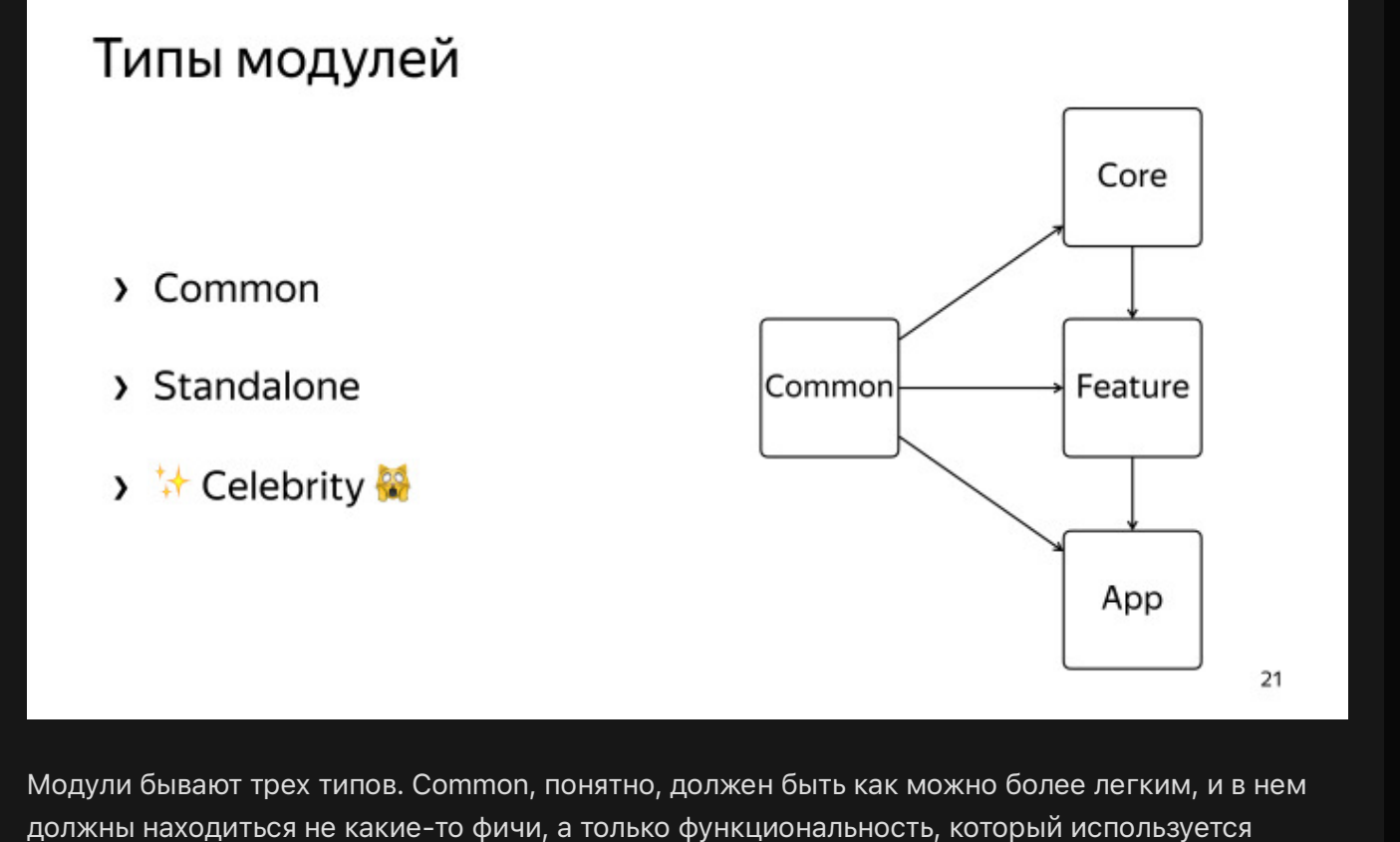

После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core
Feature
Feature
Feature
18

Другой вариант работает не всегда. Например, если это какой-то Network, то я сложно себе представляю, как это может произойти, но если это какой-то экран логина пользователя, то вполне может быть.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Core Sample App
Core
Feature
Feature
Feature
19

Можем сделать Sample, такой же полноправный корневой модуль как App, и собирать в нем только одну фичу, это будет очень быстро, и это можно быстро итеративно разрабатывать. В конце презентации покажу, сколько времени занимает обычная сборка и сборка сэмпла.

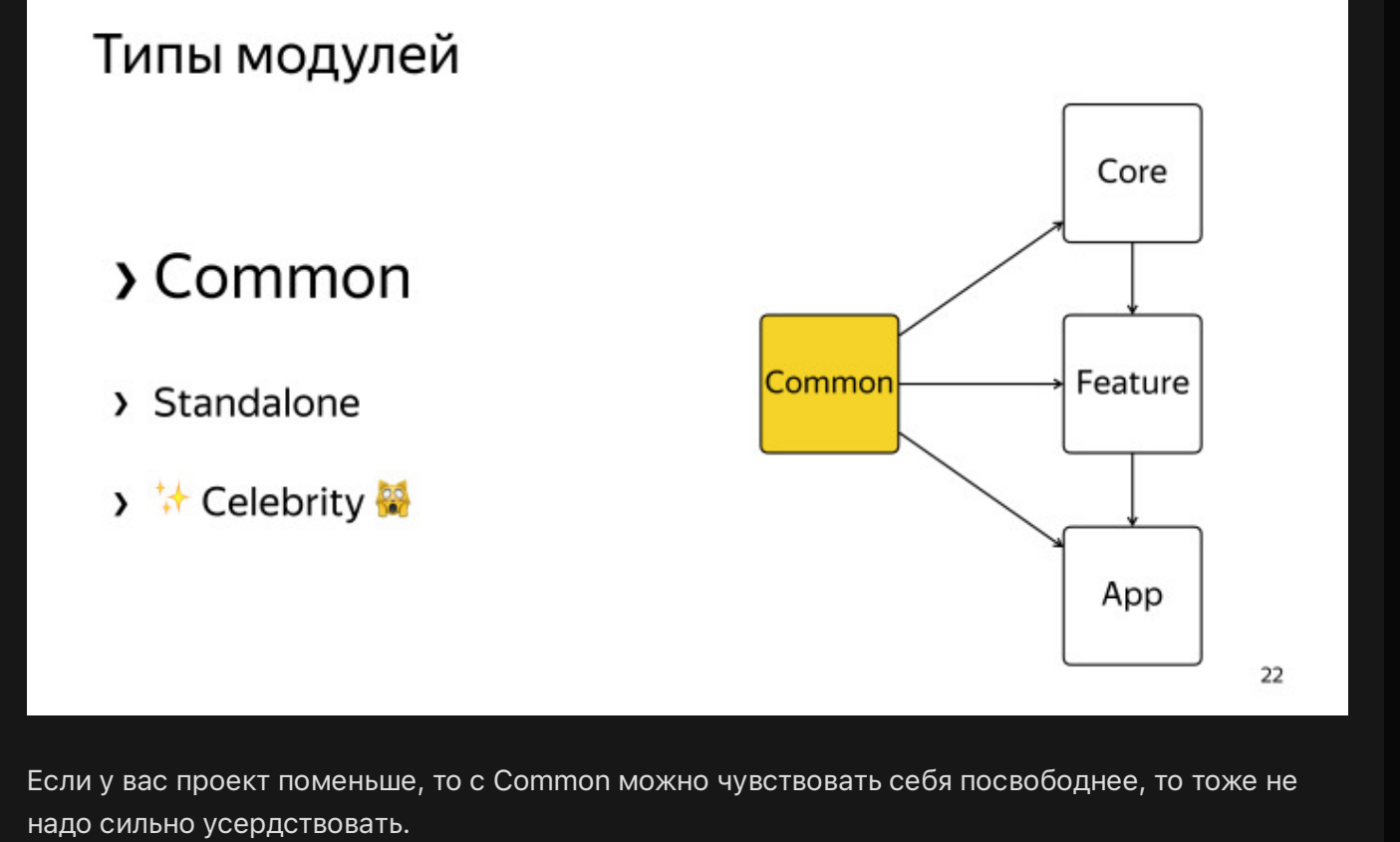
С первой частью закончили. Какие модули бывают?


Типы модулей

- Common
- Standalone
- ✨ Celebrity 🙀

Core
Common
Feature
App
21

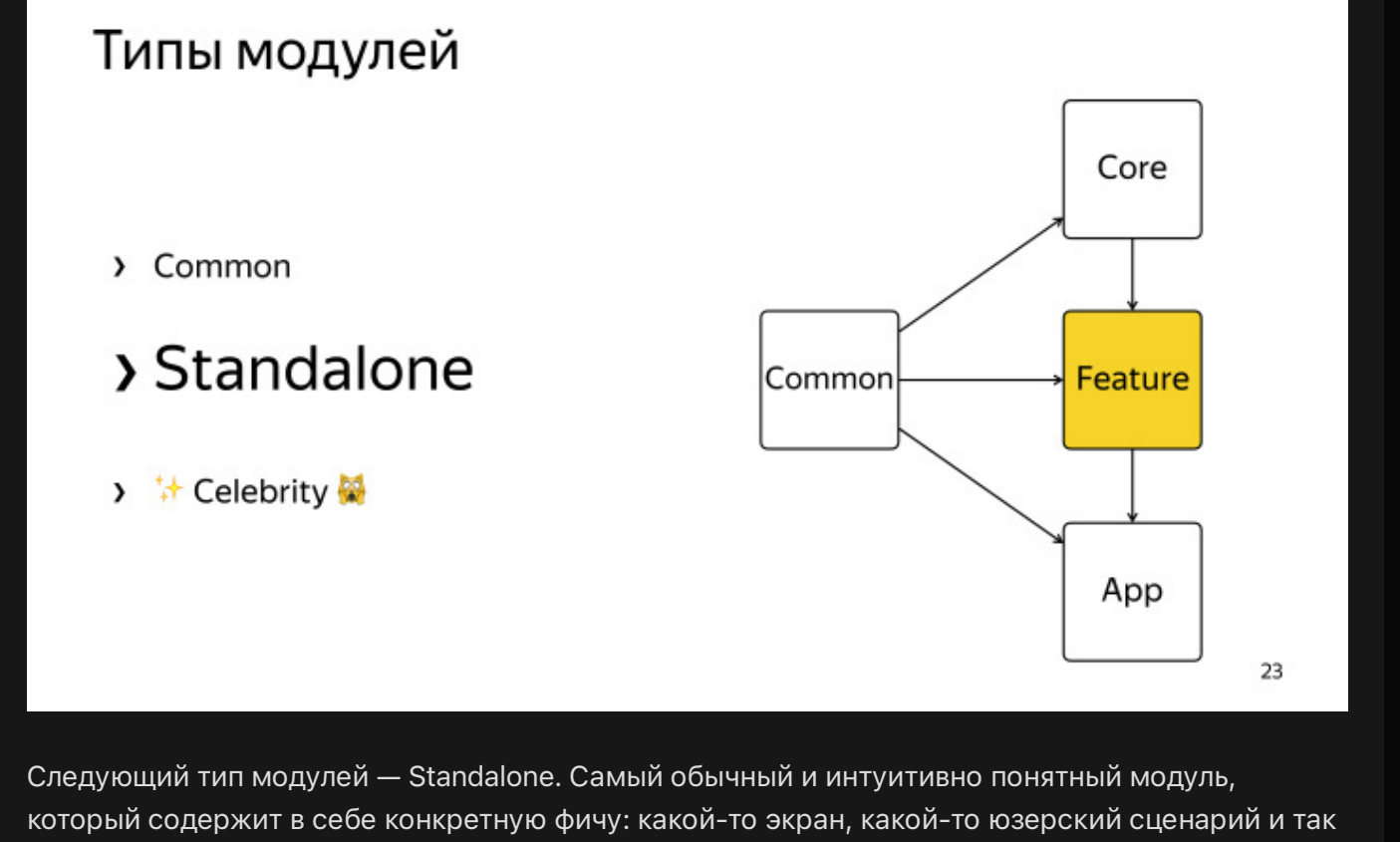
Модули бывают трех типов. Common, понятно, должен быть как можно более легким, и в нем должны находиться не какие-то фичи, а только функциональность, который используется всеми. Для нас в нашей команде это особенно важно. Если мы будем предоставлять наши Feature модули другим приложениям, то мы будем их заставлять тащить Common в любом случае. Если он будет очень жирный, то нас никто не будет любить.


Типы модулей

- Common
- Standalone
- ✨ Celebrity 🙀

Core
Common
Feature
App
22

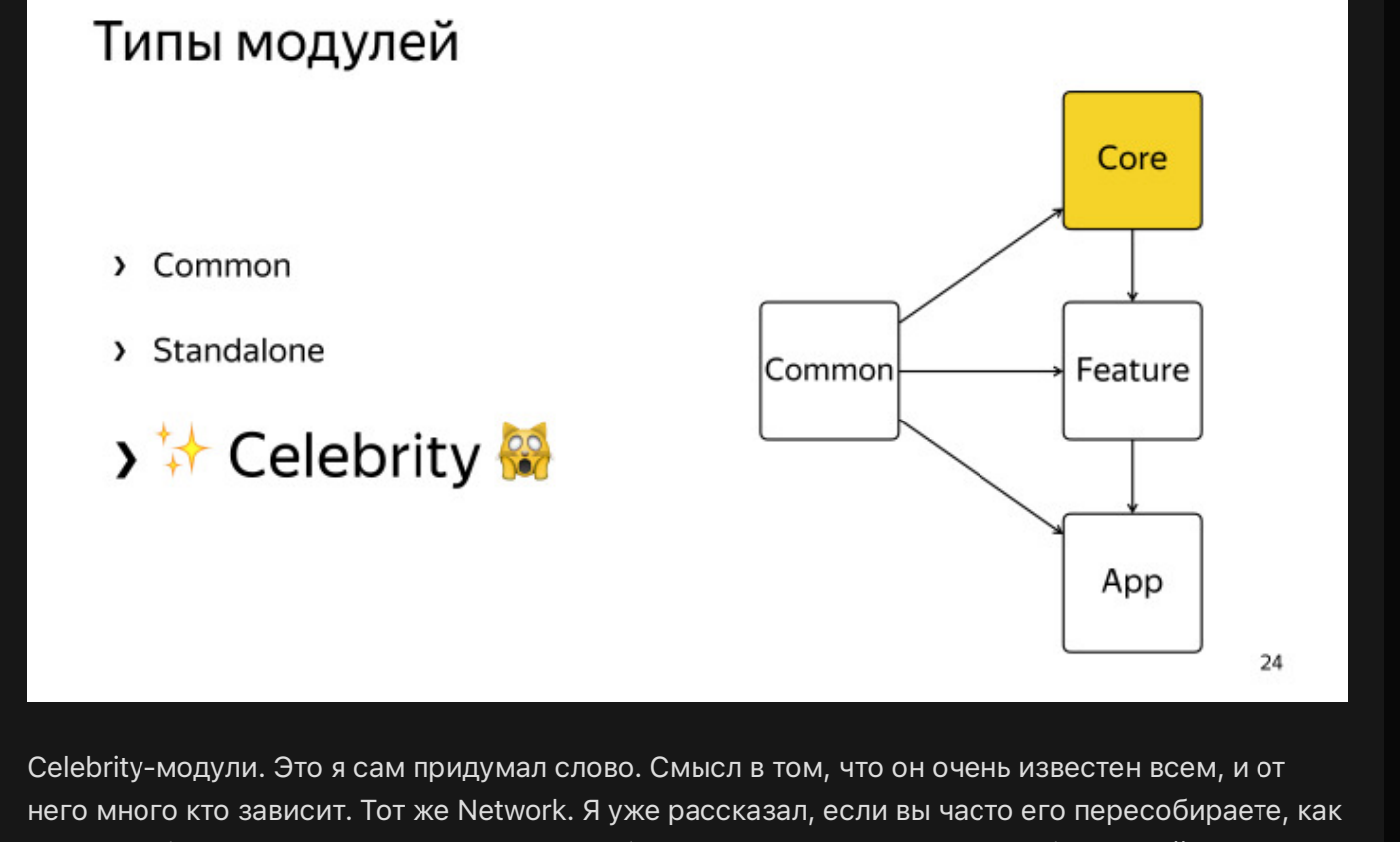
Если у вас проект поменьше, то с Common можно чувствовать себя посвободнее, то тоже не надо сильно усердствовать.


Типы модулей

- Common
- Standalone
- ✨ Celebrity 🙀

Core
Common
Feature
App
23

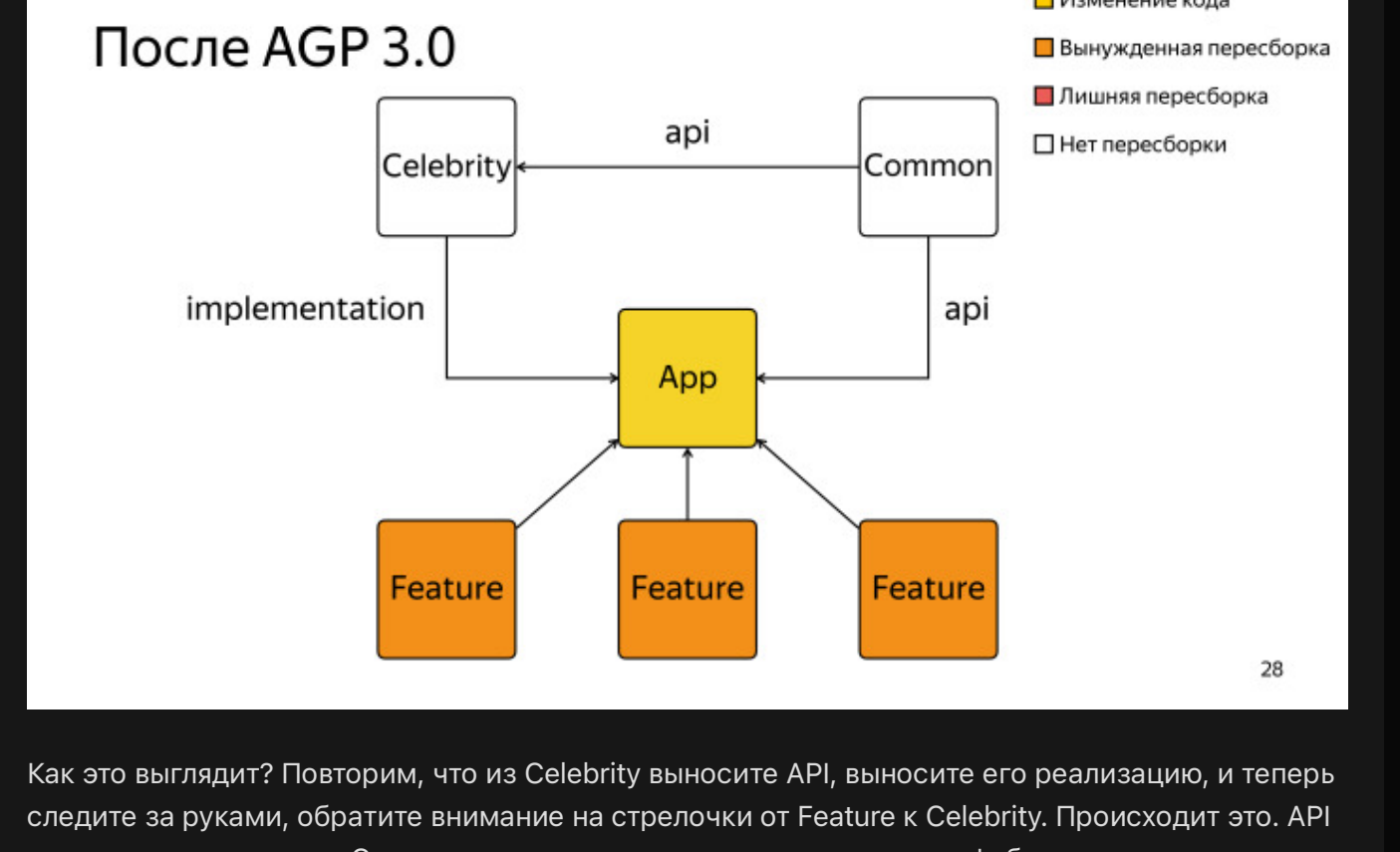
Следующий тип модулей — Standalone. Самый обычный и интуитивно понятный модуль, который содержит в себе конкретную фичу: какой-то экран, какой-то юзерский сценарий и так далее. Он должен быть по возможности независимый, и для него чаще всего можно сделать sample App и разрабатывать его в нем. Sample App очень сильно важны в начале процесса разбиения, потому что все билдится по-прежнему медленно, и вы хотите получить как можно быстрее профит. В конце, когда все будет побито на модули, вы можете пересобирать все, это будет быстро. Потому что не будет пересобираться лишний раз.


Типы модулей

- Common
- Standalone
- ✨ Celebrity 🙀

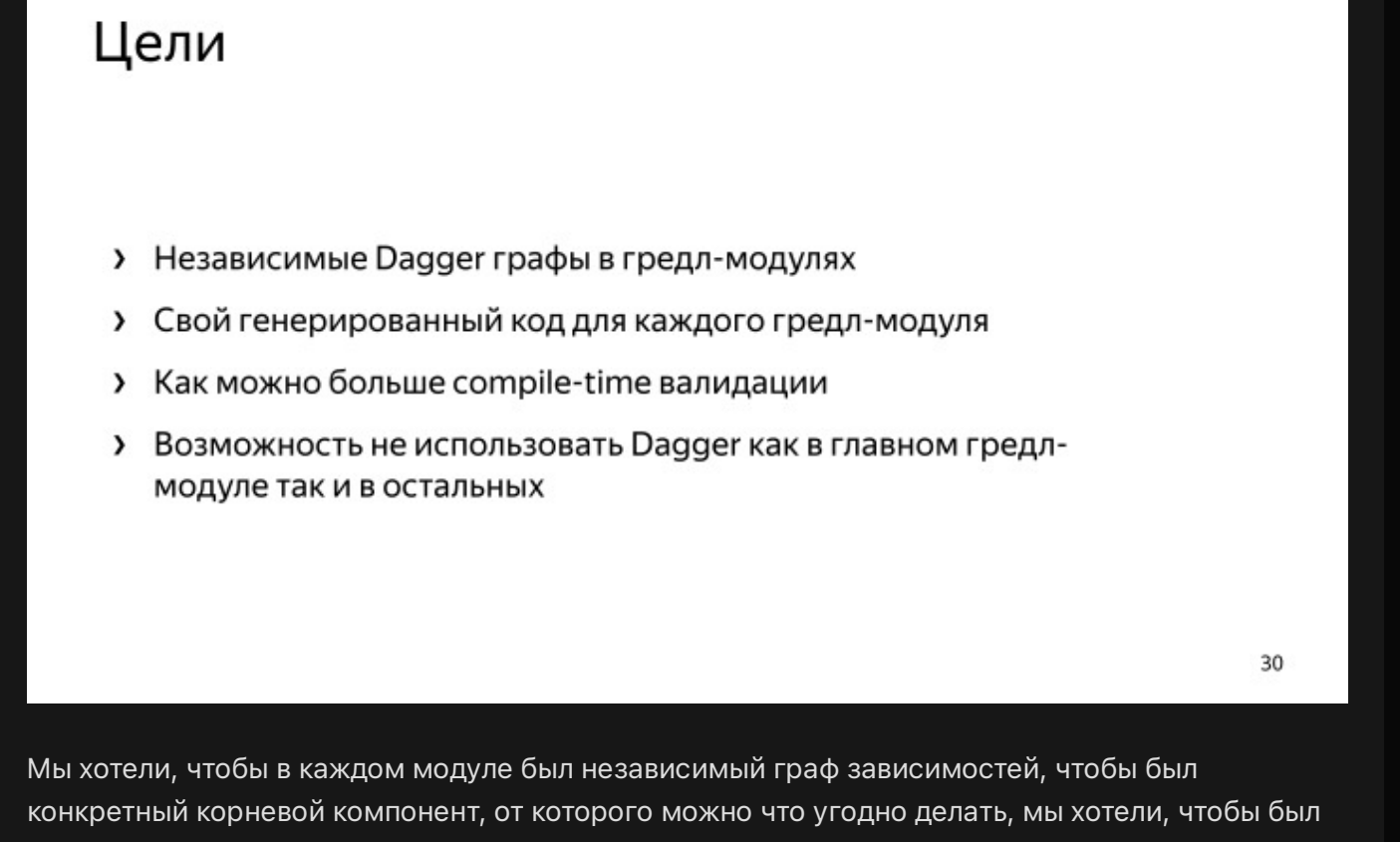
Core
Common
Feature
App
24

Celebrity-модули. Это я сам придумал слово. Смысл в том, что он очень известен всем, и от него много кто зависит. Тот же Network. Я уже рассказал, если вы часто его пересобираете, как можно избежать того, что у вас все пересобирается. Есть еще один способ, который можно применить для небольших проектов, для которых не стоит цели отдавать все наружу как отдельную зависимость, отдельный артефакт.


После AGP 3.0
🟨 Изменение кода
🟧 Вынужденная пересборка
🟥 Лишняя пересборка
☐ Нет пересборки
Celebrity
api
Common
implementation
api
App
Feature
Feature
Feature
28

Как это выглядит? Повторим, что из Celebrity выносите API, выносите его реализацию, и теперь следите за руками, обратите внимание на стрелочки от Feature к Celebrity. Происходит это. API вашего модуля попал в Common, реализация осталась сама в нем, а фабрика, которая предоставляет реализацию этого API, появилась в вашем главном модуле. Если кто-то смотрел Mobius, то про это рассказывал Денис Неклюдов. Очень похожая схема.

Мы в проекте используем Dagger, нам он нравится, и мы хотели как можно больше от этого пользы получить в контексте разных модулей.


Цели

- Независимые Dagger графы в гредл-модулях
- Свой генерированный код для каждого гредл-модуля
- Как можно больше compile-time валидации
- Возможность не использовать Dagger как в главном гредл-модуле так и в остальных

Мы хотели, чтобы в каждом модуле был независимый граф зависимостей, чтобы был конкретный корневой компонент, от которого можно что угодно делать, мы хотели, чтобы был свой сгенерированный код для каждого Gradle-модуля. Мы не хотели, чтобы сгенерированный код пролезал в главный. Мы хотели как можно больше compile-time валидации. Мы же страдаем от [k]apt, хоть какой-то профит должны получить от того, что дает Dagger. И при всем этом мы не хотели заставлять никого использовать Dagger. Ни того, кто реализует свежий фиче-модуль отдельный, ни того, кто его потом потребляет, наши коллеги, которые просят какие-то фичи себе.

Как организовать отдельный граф зависимостей внутри нашего фиче-модуля?

@Subcomponent?

ИНФОРМАЦИЯ

Сайт www.ya.ru
Дата регистрации 9 августа 2008
Дата основания 23 сентября 1997
Численность свыше 10 000 человек
Местоположение Россия

ВИДЖЕТ

Все хабростатьи Яндекса в одном ТГ-канале

БЛОГ НА ХАБРЕ

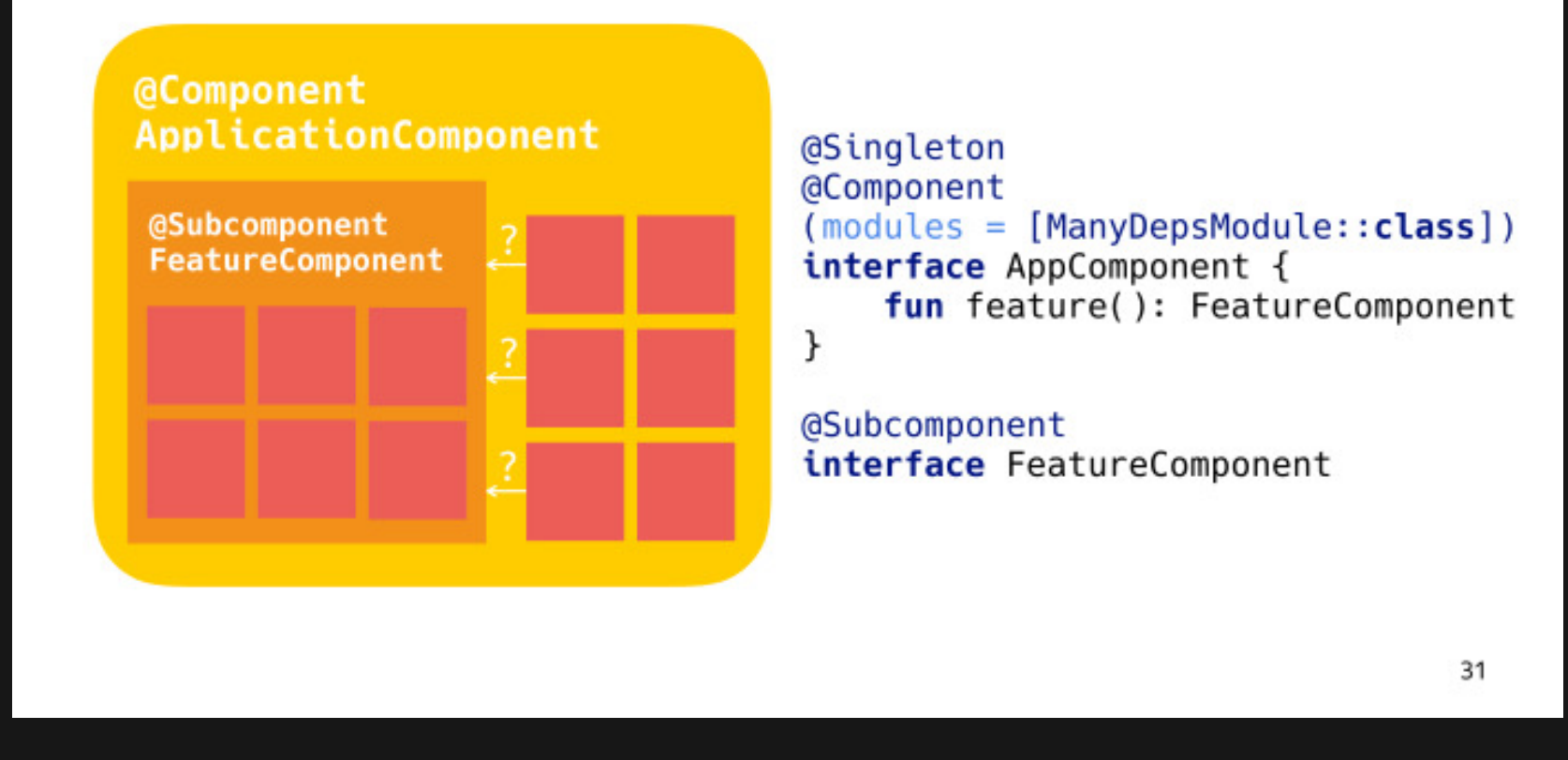
11 апр в 12:00
Как коммитить так, чтобы ваш код принимали мейнтейнеры: путь одного героя
3.4K 1 +1

9 апр в 13:00
Магия персональных рекомендаций, или как нейросеть Яндекс Карт подбирает места под интересы пользователей
3.2K 13 +13

7 апр в 10:00
Сбой с вероятностью один раз в 20 лет: о мартовском инциденте в дата-центре
38K 156 +156

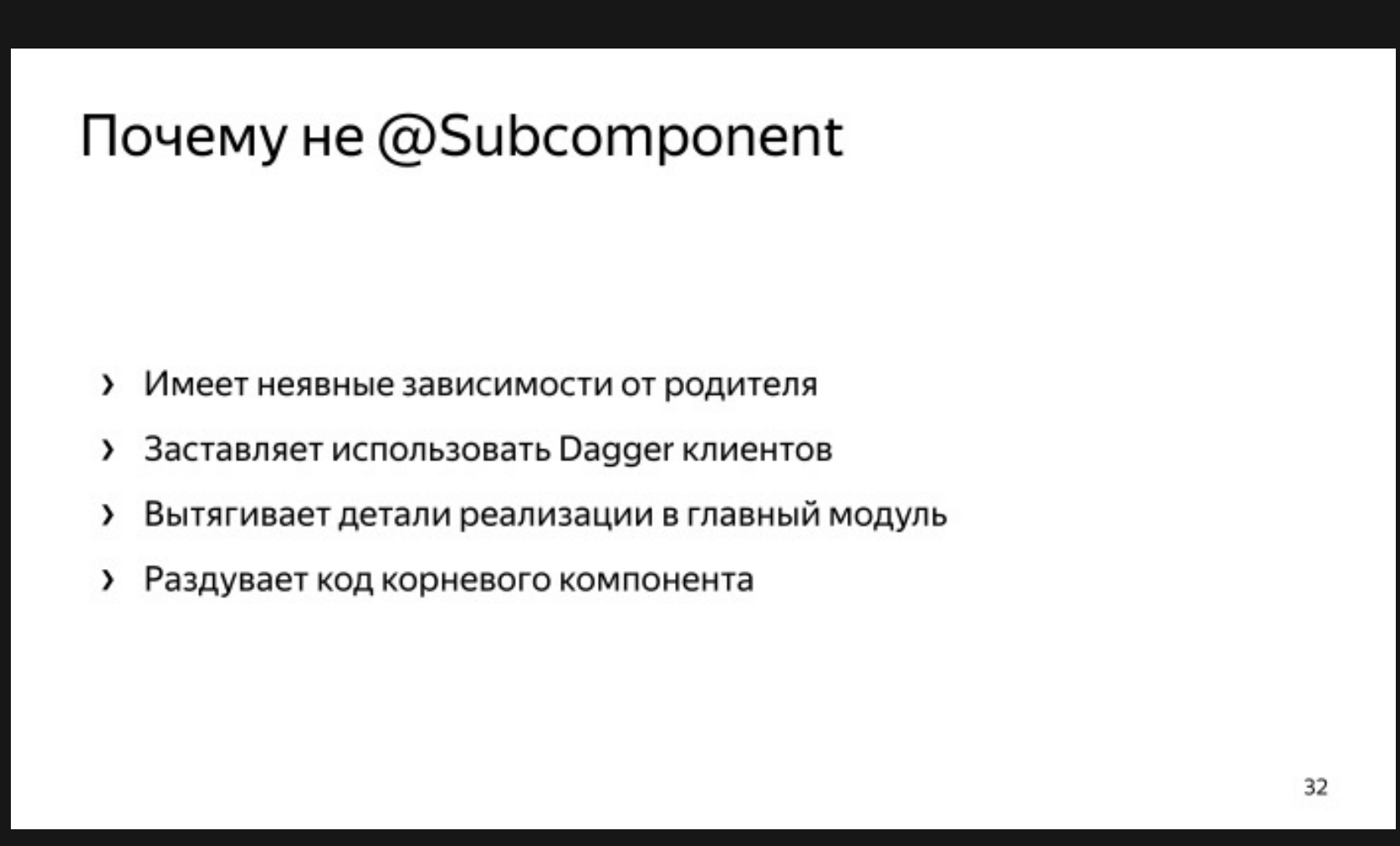
2 апр в 12:00
Пишем свой pastebin, используя только userver
3.6K 9 +9

31 мар в 10:04
Открываем instruct-версию YandexGPT 5 Lite
8K 20 +20



```
@Singleton
@Component
(modules = [ManyDepsModule::class])
interface AppComponent {
    fun feature(): FeatureComponent
}

@Subcomponent
interface FeatureComponent
```

Можно попытаться использовать Subcomponent, и это даже будет работать. Но у этого довольно много недостатков. Можно увидеть, что в Subcomponent непонятно, какие именно зависимости он использует из Component. Чтобы это понять, вам придется долго и мучительно пересобирать проект, смотреть, на что ругается Dagger и его добавлять.
К тому же сабкомпоненты устроены так, что заставляют других использовать Dagger, и не получится все это легко запустить у ваших клиентов и вас самих, если вы решите вдруг в каком-то модуле отказаться.

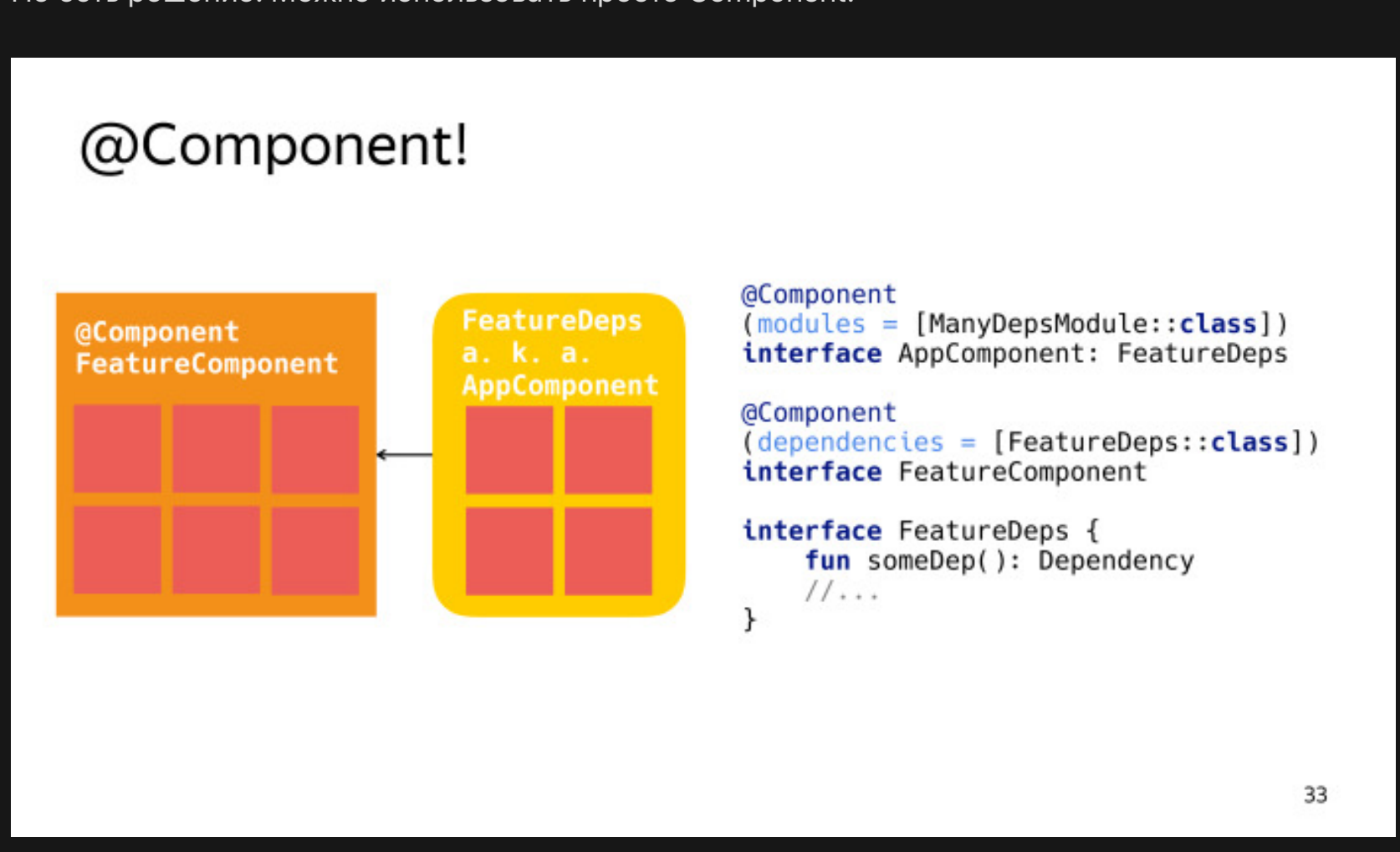


Почему не @Subcomponent

- Имеет неявные зависимости от родителя
- Заставляет использовать Dagger клиентов
- Вытягивает детали реализации в главный модуль
- Раздувает код корневого компонента

Одна из самых отвратительных вещей, что при использовании Subcomponent, все зависимости вытягиваются в главный модуль. Dagger устроен так, что сабкомпоненты генерятся вложенным классом их обрамляющих компонентов, родительских. Может, кто-то смотрел генеренный код и его размер, на свои генеренные компоненты? У нас 20 тыс. строчек в нем. Так как сабкомпоненты всегда вложенные классы для компонентов, то получается, что сабкомпоненты сабкомпонентов тоже вложенные, и весь генеренный код попадает в главный модуль, этот двадцатитысячестрочный файл, который нужно компилить, и нужно его рефакторить, Студия начинает тормозить — боль.

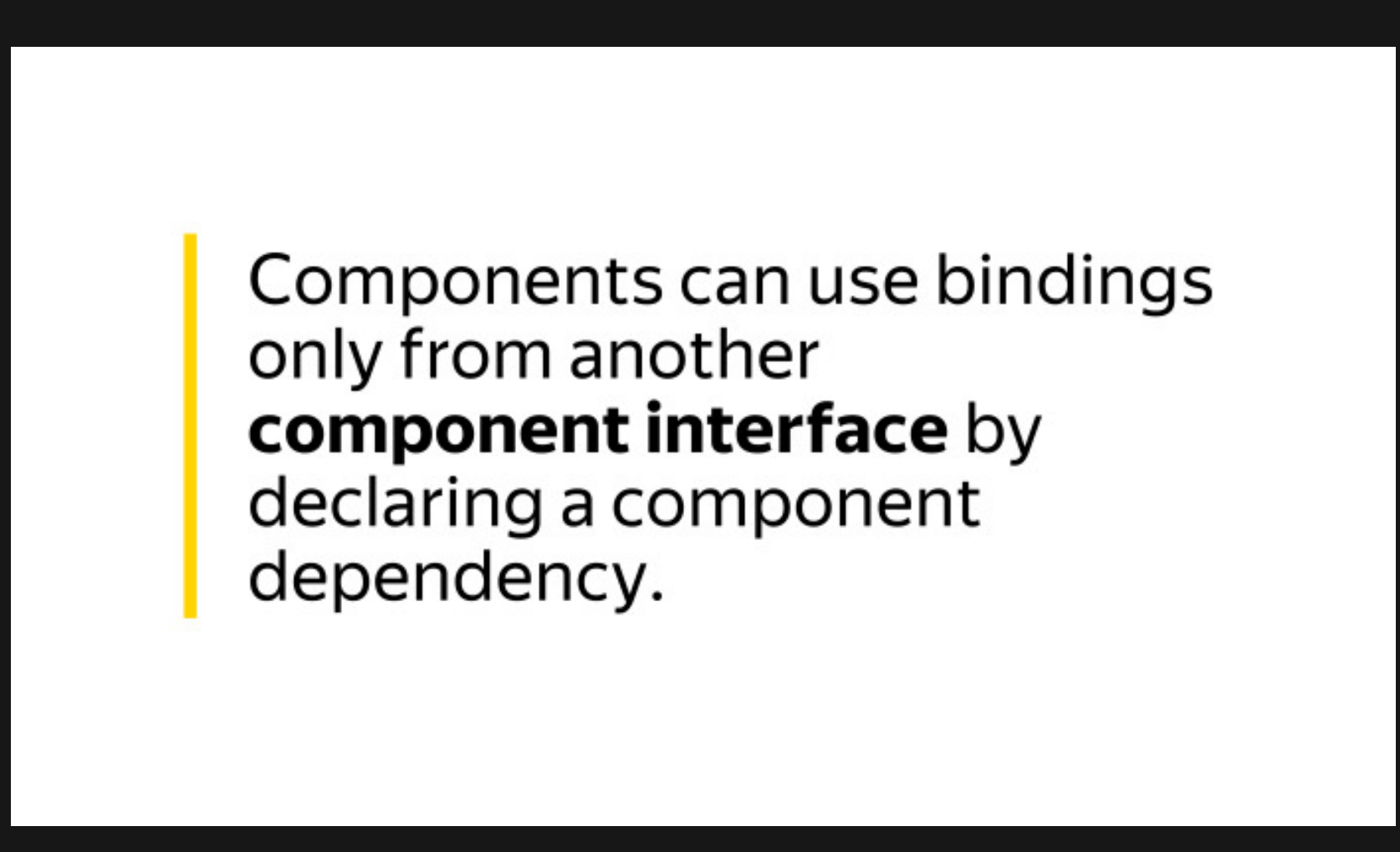
Но есть решение. Можно использовать просто Component.



@Component!

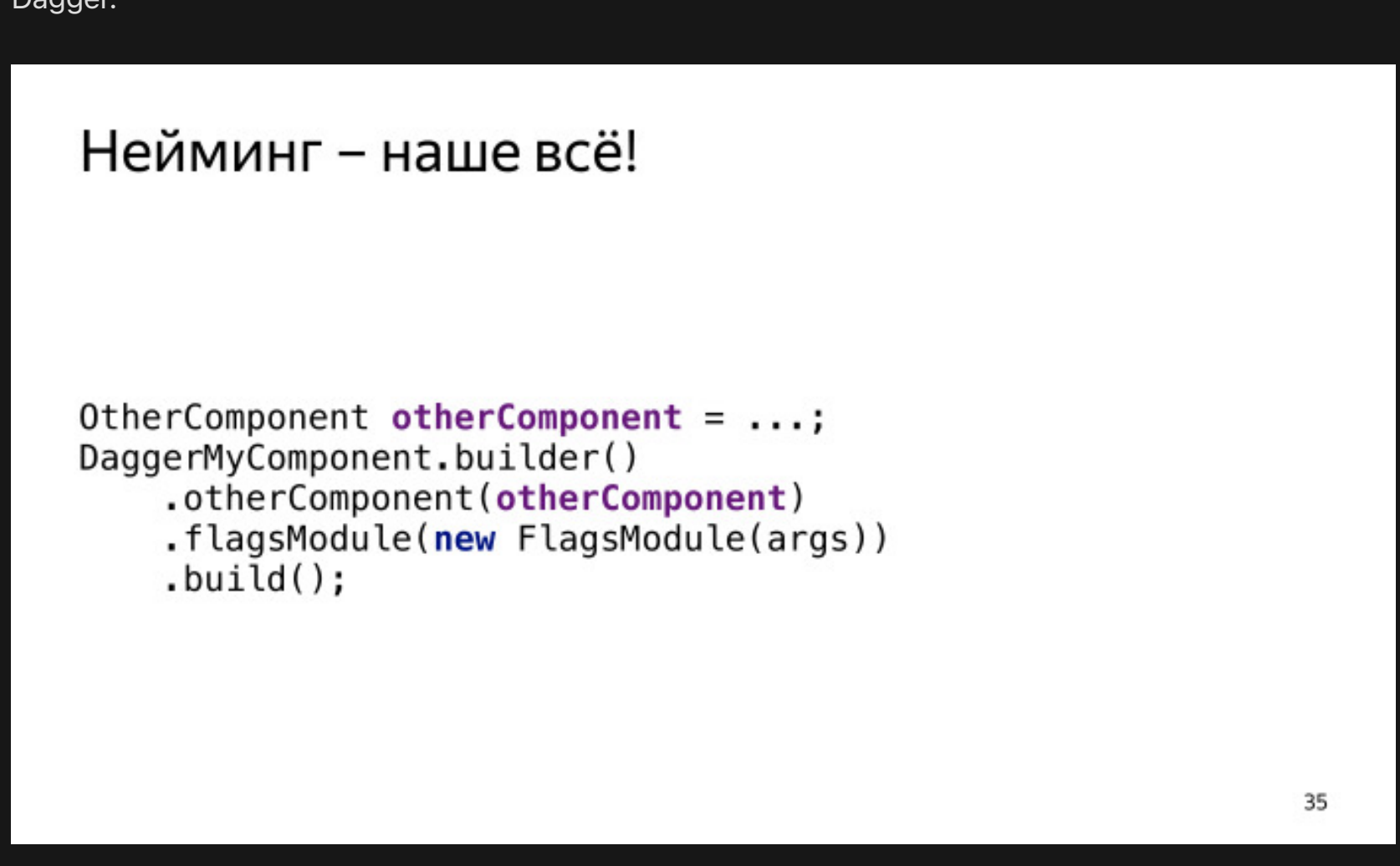
```
@Component
(modules = [ManyDepsModule::class])
interface AppComponent: FeatureDeps

@Component
(dependencies = [FeatureDeps::class])
interface FeatureComponent

interface FeatureDeps {
    fun someDep(): Dependency
    //...
}
```

В Dagger можно компоненту указывать зависимости. Это показано в коде, и показано на картинке. Зависимости, где вы указываете Provision методы, фабричные методы, которые показывают, от каких именно сущностей зависит ваш компонент. Он хочет их получить в момент создания.
Раньше я всегда думал, что в эти зависимости можно указывать только другие компоненты, и вот почему — в документации написано так.



> Components can use bindings only from another **component interface** by declaring a component dependency.

Теперь я понимаю, что значит использовать component interface, но раньше я думал, что это просто компонент. На самом деле нужно использовать интерфейс, который составлен по правилам создания интерфейса для компонента. Короче говоря, просто Provision-методы, когда у вас просто есть геттеры для каких-то зависимостей. Также пример кода есть в документации Dagger.

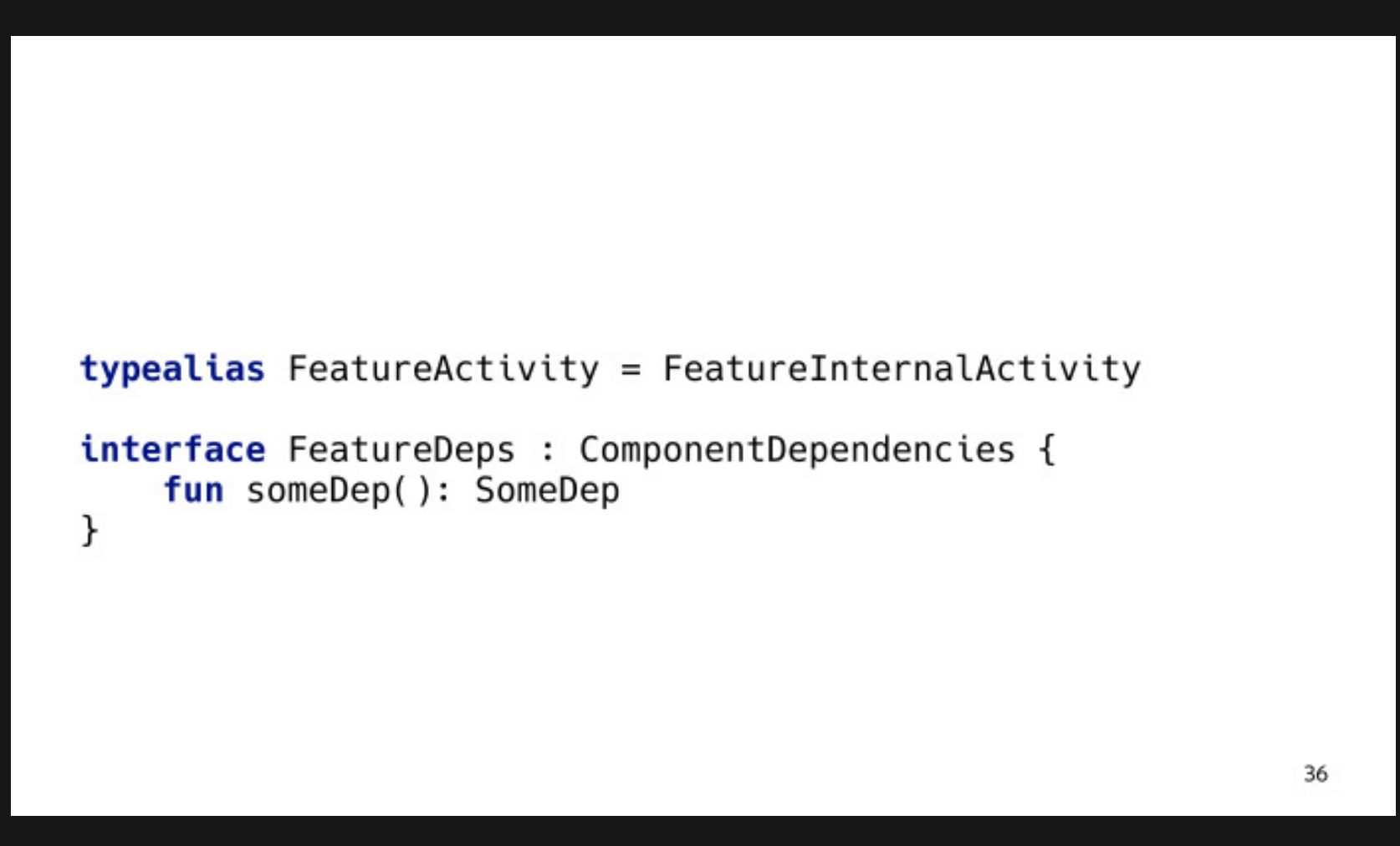


Нейминг – наше всё!

```
OtherComponent otherComponent = ...;
DaggerMyComponent.builder()
    .otherComponent(otherComponent)
    .flagsModule(new FlagsModule(args))
    .build();
```

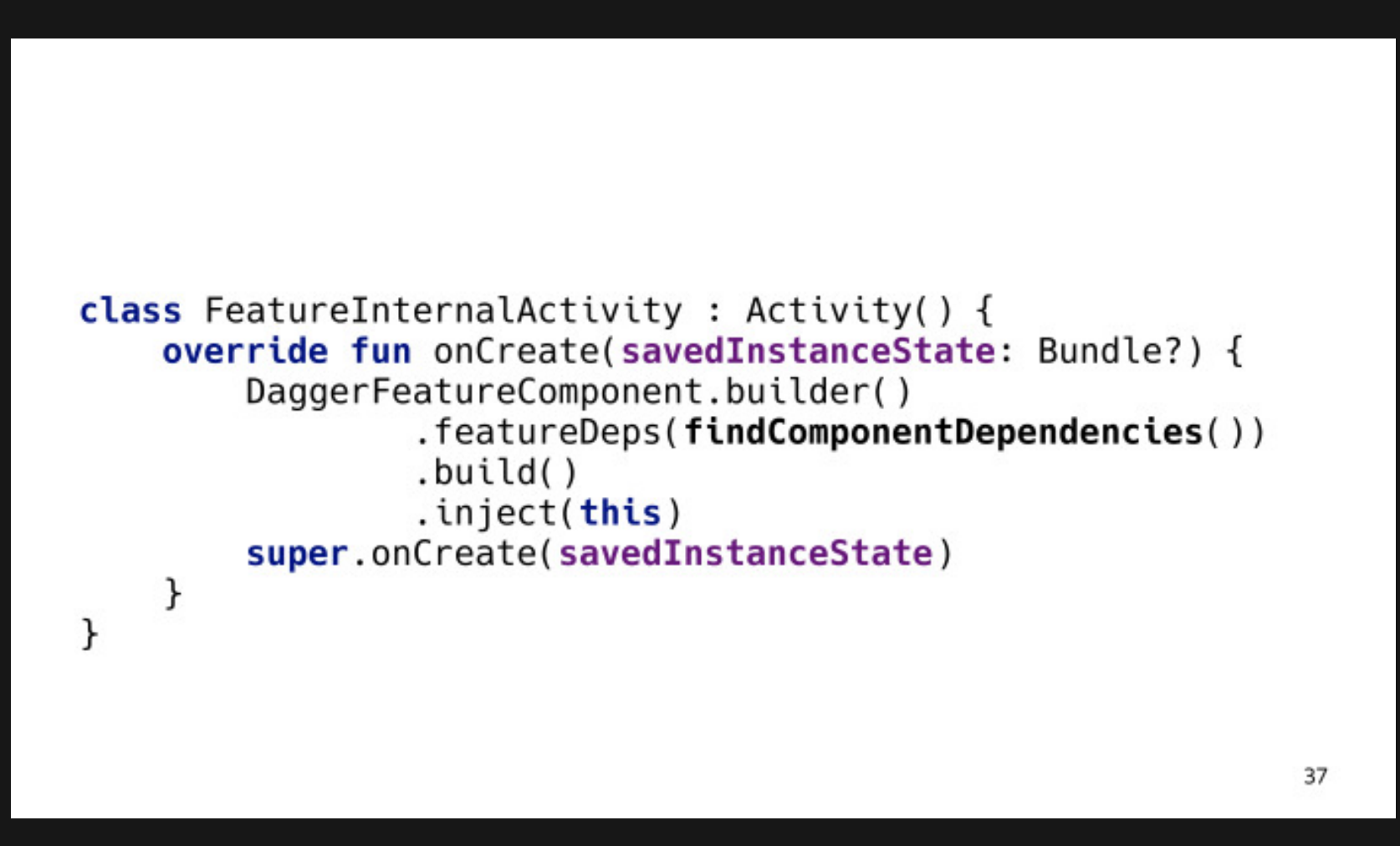
Там тоже написано otherComponent, и это сбивает с толку, потому что на самом деле туда можно не только компоненты засовывать.

Как бы нам хотелось это дело использовать в реальности?



```
typealias FeatureActivity = FeatureInternalActivity

interface FeatureDeps : ComponentDependencies {
    fun someDep(): SomeDep
}
```

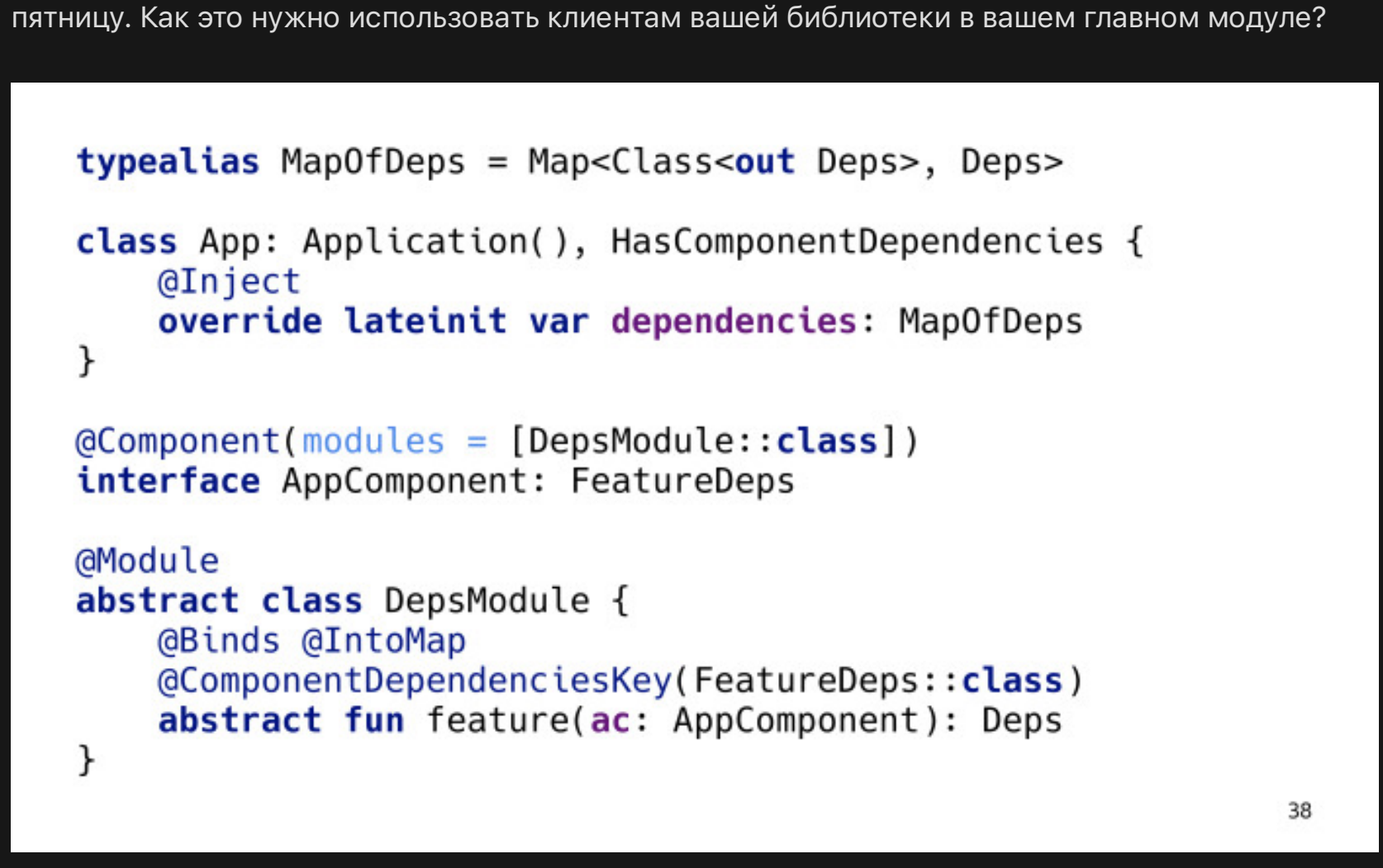
В реальности есть Feature-модуль, у него есть пакет API, который виден, находится близко к корню всех пакетов, и там указано, что есть точка входа — FeatureActivity. Необязательно использовать typealias, просто чтобы было понятно. Это может быть фрагмент, может быть ViewController — неважно. И есть его зависимости, FeatureDeps, где указано, что ему нужен контекст, какой-то Network-сервис, из Common какая-то штука, которую хочется получать из App, и любой клиент обязан это удовлетворить. Когда он это сделает, все заработает.



```
class FeatureInternalActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        DaggerFeatureComponent.builder()
                .featureDeps(findComponentDependencies())
                .build()
                .inject(this)
        super.onCreate(savedInstanceState)
    }
}
```

Как мы всем этим пользуемся в Feature-модуле? Здесь я использую Activity, это необязательно. Мы как обычно создаем свой корневой Dagger-компонент и используем волшебный метод findComponentDependencies, он сильно похож на Dagger for Android, но мы его не можем использовать в первую очередь потому, что не хотим тащить сабкомпоненты. В остальном всю логику мы можем у них перенять.

Я сначала пытался рассказать, как он работает, но вы это сможете увидеть в семпл-проекте в пятницу. Как это нужно использовать клиентам вашей библиотеки в вашем главном модуле?



```
typealias MapOfDeps = Map<Class<out Deps>, Deps>

class App: Application(), HasComponentDependencies {
    @Inject
    override lateinit var dependencies: MapOfDeps
}

@Component(modules = [DepsModule::class])
interface AppComponent: FeatureDeps

@Module
abstract class DepsModule {
    @Binds @IntoMap
    @ComponentDependenciesKey(FeatureDeps::class)
    abstract fun feature(ac: AppComponent): Deps
}
```

В первую очередь это просто typealias. На самом деле он имеет другое название, но для краткости так. MapOfDepth по классу интерфейса Dependency отдает вам его реализацию. В App говорим, что мы умеем проводить зависимости так же, как в Dagger for Android, и очень важно, что компонент наследует этот интерфейс, автоматически получает Provision-методы. Dagger с этого момента начинает нас заставлять предоставить эту зависимость. Пока вы ее не предоставите, он не будет компилиться. В этом главное удобство: вы решили устроить фичу, расширили свой компонент этим интерфейсом — все, пока вы не сделаете все остальное, он не будет просто компилиться, а будет выдавать понятные сообщения об ошибках. Модуль простой, смысл в том, что он биндит ваш компонент к реализации интерфейса. Примерно так же, как в Dagger for Android.

Перейдем к результатам.

Я проверял на нашем мейнфрейме и на моем локальном ноутбуке, перед этим выключив все что можно. Если мы добавляем публичный метод в Feature, то время билда значительно отличается. Здесь я показываю различия в случае, когда я билжу семпл-проект. Это 16 секунд. Или когда собираю все карты — это значит две минуты сидеть и ждать при каждом, даже минимальном изменении. Поэтому многие фичи мы разрабатываем и будем разрабатывать в семпл-проектах. На мейнфрейме время сопоставимое.

Еще один важный результат. До выделения модуля Feature это выглядело так: на мейнфрейме было 28 секунд, теперь стало 49 секунд. Мы выделили первый модуль, и уже получили замедление сборки чуть ли не в два раза.

И еще один вариант — простая инкрементальная сборка нашего модуля, не фичи, как в предыдущем. 28 секунд было до выделения модуля. Когда выделили код, который не нужно каждый раз пересобирать, и [k]apt, который не нужно каждый раз проводить, — выиграли три секунды. Не бог весть что, но надеюсь, с каждым новым модулем время будет только уменьшаться.

Вот полезные ссылки на статьи: API против implementation, статья с замерами времени билда, семпл-модуль. Презентация будет доступна. Спасибо.

Теги: dagger 2, dependency injection, multimodality

Хабы: Блог компании Яндекс, Разработка мобильных приложений, Разработка под Android

+12 74 2 +2

Редакторский дайджест
Присылаем лучшие статьи раз в месяц

Электропочта

Яндекс
Как мы делаем Яндекс
Github

216 Карма 0 Рейтинг

Леонид Клюев @Leono
Интернет щей

Подписаться

Комментарии 2

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

stas_dubich 21 час назад
Проксирование из коробки: сравнительный анализ HAProxy, Envoy, Nginx, Caddy и Traefik
Средний 18 мин 7.1K
Из песочницы
+44 84 21 +21

techno_mot 4 часа назад
DNS: поддельные записи, перехват трафика и другие ужасы
14 мин 2.1K
+31 28 0

valisak 23 часа назад
Вопросы о тёмной энергии: нужна ли она, можно ли заменить её чёрными дырами и меняется ли она со временем
Средний 9 мин 3.3K
Обзор
+25 13 18 +18

sec_urist 8 часов назад
GrapheneOS: Pixel, но не Google
Простой 16 мин 7.6K
Из песочницы
+23 27 7 +7

alexgreendev 7 часов назад
Как ИИ не заменит программистов, а создаст дефицит кадров в IT больше, чем COVID-19
Простой 7 мин 14K
Мнение
+22 23 35 +35

noobaitranslator 18 часов назад
Google инструкция по промпт инжинирингу или как правильно писать запросы (краткий перевод)
Простой 8 мин 10K
Из песочницы Перевод
+19 125 7 +7

ulechka 20 часов назад
Какие функции родительского контроля полезны на каждом этапе развития вашего ребенка (мнение родителя)
11 мин 1.9K
Мнение
+17 26 17 +17

VBart 34 минуты назад
Что нового в Angie 1.9 и что ожидать от 1.10?
Простой 9 мин 201
Обзор
+16 1 0

OlegIct 20 часов назад
Визуальное представление структуры btree индекса PostgreSQL
Средний 16 мин 4.1K
Туториал
+16 79 0

SLY_G 4 часа назад
Микрокомпьютеры — первая волна: реакция на Altair
Простой 19 мин 767
Перевод
+14 4 0

Показать еще

Ваш аккаунт
- Войти
- Регистрация

Разделы
- Статьи
- Новости
- Хабы
- Компании
- Авторы
- Песочница

Информация
- Устройство сайта
- Для авторов
- Для компаний
- Документы
- Соглашение
- Конфиденциальность

Услуги
- Корпоративный блог
- Медийная реклама
- Нативные проекты
- Образовательные программы
- Стартапам

© 2006–2025, Habr

Техническая поддержка

Настройка языка